

RESEARCH ARTICLE

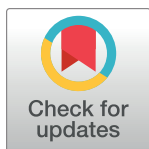
TurboBlom: A light and resilient key predistribution scheme with application to Internet of Things

Majid Khabbazzian^{1*}, Reihaneh Safavi-Naini², Ahmad Shabani-Baghani¹

¹ Department of Electrical and Computer Engineering, University of Alberta, Edmonton, Alberta, Canada,

² Department of Computer Science, University of Calgary, Calgary, Alberta, Canada

* mkhabbazzian@ualberta.ca



OPEN ACCESS

Citation: Khabbazzian M, Safavi-Naini R, Shabani-Baghani A (2024) TurboBlom: A light and resilient key predistribution scheme with application to Internet of Things. PLoS ONE 19(3): e0295190. <https://doi.org/10.1371/journal.pone.0295190>

Editor: Vincent Omollo Nyangaresi, Jaramogi Oginga Odinga University of Science and Technology, KENYA

Received: July 4, 2023

Accepted: November 16, 2023

Published: March 20, 2024

Copyright: © 2024 Khabbazzian et al. This is an open access article distributed under the terms of the [Creative Commons Attribution License](https://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Data Availability Statement: All relevant data are within the paper.

Funding: This article was funded by the Natural Sciences and Engineering Research Council of Canada (NSERC). URL: https://www.nserc-crsng.gc.ca/index_eng.asp The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript. The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

Abstract

In the Internet of Things (IoT), there are often devices that are computationally too constrained to establish a security key using traditional key distribution mechanisms such as those based on the Diffie-Hellman key exchange. To address this, current solution commonly rely on key predistribution schemes (KPSs). Among KPSs, the Blom scheme provably provides the highest resilience against node capture attacks. This, however, comes at high computational overhead, because the Blom scheme requires many multiplications over a large finite field. To overcome this computational overhead, we present TurboBlom, a novel amendment of the Blom scheme. TurboBlom circumvents the need for field multiplications by utilizing specialized generator matrices, such as random zero-one matrices. We demonstrate that, through this approach, TurboBlom can significantly reduce the computational overhead of the Blom scheme by orders of magnitude. In our next key finding, we demonstrate that TurboBlom offers a level of resilience against node capture that is virtually on par with the Blom scheme. Notably, we prove that the gap between the resilience of the two schemes is exponentially small. These features of TurboBlom (i.e., low computational overhead and high resilience) make it suitable for computationally constrained devices. Such devices exist in abundance in IoT, for example, as part of Low Power and Lossy Networks (LLNs). To demonstrate a sample application of TurboBlom, we show how to use it to enable sender authentication in the Routing Protocol for LLNs (RPL), a standard routing protocol for IoT.

1 Introduction

To ensure the confidentiality and integrity of data exchanged, it is crucial that network devices are able to establish security keys between themselves. These keys are used to safeguard the privacy of user information and prevent various attacks such as protocol manipulation [1], and guarantee the integrity of the data [2]. In networks such as LLNs and IoT, however, there are often devices that are computationally too constrained to establish a security key using traditional key distribution mechanisms such as those based on the Diffie-Hellman key exchange.

Competing interests: The authors have declared that no competing interests exist.

These devices include Class 1 of constrained devices defined in RFC 7228 [3]. Devices in this class are capable enough to run a protocol stack specifically designed for constrained nodes (e.g., the Constrained Application Protocol (CoAP) over UDP [4]) and communicate with nodes employing a full protocol stack with the help of a gateway. They are, however, unable to implement a full protocol stack such as Transport Layer Security (TLS), and related security protocols according to [3].

Given enough time, devices in higher classes in RFC 7228 have the potential to run traditional key exchange schemes. For example, experimental results in [5] show that their proposed Diffie-Hellman Protocol requires, on average, 54 seconds to establish a key between two TelosB IoT devices. It's important to note that the higher execution time of traditional key exchange schemes in these devices exposes them to denial of service attacks where, for instance, an adversary would consume their time/power resources by repeatedly initiating key exchanges with them.

An alternative to the traditional key distribution schemes is key predistribution scheme (KPS), commonly considered for networks with constrained devices [6, 7]. In a KPS, a trusted entity preloads each device/node with one or more secret keys before the node is deployed in the network. After deployment, a pair of nodes in the network use their preloaded secret keys to establish a pairwise key and consequently secure their communications.

The simplest key predistribution strategy is to use a single shared key over the entire network. This scheme is the most efficient KPS in terms of memory usage, as each node stores only a single secret key. On the negative side, the scheme is the most vulnerable KPS to node corruption as a single corrupted node (e.g., a node captured and compromised by an adversary) compromises the security of the entire network, as the entire network uses one identical security key.

An alternative strategy is to preload each node with many secret keys; one key for every other node in the network. This way, any node can securely communicate with any other node in the network using a unique secret key. This KPS is the most resilient scheme against node corruption/capture. The scheme is, however, the least efficient with regards to memory (memory requirement linearly increases with the number of nodes in the network). In addition, the scheme requires every existing node to securely receive a new secret key each time a new node is added to the network (the new keys are needed for secure communication of the existing nodes with the newly added node). The above shortcomings are not desired particularly in large-scale networks.

There are “middle-ground” KPS solutions as well. For instance, in random key predistribution schemes (e.g., [8, 9]), each node is preloaded with a random subset of keys from a key pool. To generate a pairwise key, two deployed nodes exchange the indices of their keys to discover whether or not they share any keys. If the two nodes share at least one key, they use the common key(s) to generate a pairwise key. These solutions are memory-friendly, and are resilient against node capture.

Similar to the random KPSs, the Blom scheme [10] (which is the focus of this work) preloads each node with a number of secret keys. It, however, uses a deterministic key generation scheme, and brings two main advantages over random KPSs. First, it enables any two nodes in the network to generate a pairwise key with probability 100%. Second, and perhaps more importantly, the Blom scheme provides the highest resilience against node capture in the sense that the network remains completely secure as long as the number of compromised nodes is less than the number of secret keys preloaded in each node. For instance, if each node is preloaded with 200 secret keys, the Blom scheme guarantees that an adversary who has captured and extracted all the secret keys of up to 199 nodes gains no information about the pairwise keys between uncaptured nodes. This fully discourages/defends against small-scale attacks,

which is important as small-scale attacks are less expensive and harder to detect than large-scale attacks [9]. We also note that in the Blom scheme, when a new node is added to the network, the existing nodes do not need to receive any additional secret keys. As will be described in Section 3, this is because two nodes only require to know their public information in order to establish a pairwise key [10].

To put the above advantages into perspective, consider for instance the q -composite key scheme, a random KPS designed to improve resiliency against small-scale attacks [9]. Suppose each node is preloaded with 200 keys, and that the probability that two nodes being able to generate a pairwise key is set to 50% (this is a parameter in the q -composite key scheme). Then, as demonstrated in [9], with probability of about 57% the communication between two uncaptured nodes is compromised if an adversary captures 150 nodes. The resilience of the q -composite key scheme becomes even worse if we increase the probability that two nodes are being able to generate a pairwise key. In the Blom scheme, on the other hand, any two nodes are able to generate a pairwise key with probability 100%. Moreover, all the communications between uncaptured nodes remain secure if an adversary captures 150 nodes.

A limiting factor of the Blom scheme for computationally constrained devices is that its pairwise key generation requires many field multiplications. To address this, in this work, we introduce TurboBlom, our fast variant of the Blom scheme that does not require any field multiplications, hence is significantly faster (e.g., about three orders of magnitude faster as shown in Section 4) than the Blom scheme.

Despite being significantly lighter than the Blom scheme, we analytically show that TurboBlom has nearly the same level of resilience against node capture as the original Blom scheme. In particular, we show that the gap between the resilience of TurboBlom and Blom can be made exponentially small with respect to the number of pre-loaded keys. Consequently, when the number of preloaded keys is large enough (e.g. larger than 30), TurboBlom provides practically the same level of resilience against node capture as the Blom scheme itself. In addition, in Section 4, we propose techniques based on the use of finite fields of Mersenne prime order to improve resilience of TurboBlom, which is particularly helpful when the number of preloaded keys is rather small.

Finally, we demonstrate how to use TurboBlom to provide efficient (in terms of memory and computation) and resilient (against node capture and Sybil attack) sender authentication in RPL [11]. RPL is the standard routing protocol for LLNs. These networks are a major component of IoT, and have a wide scope of applications including industrial monitoring and building automation. Consequently, the security of these networks is of great importance, especially when they are deployed in critical infrastructures such as smart grids [12].

In summary, we make the following contributions:

- We introduce TurboBlom, a fast variant of the Blom scheme, which is significantly lighter than the original Blom scheme, hence is more suitable for constrained devices. The lightness of TurboBlom is specially appreciated when we note that a key exchange/distribution scheme with non-negligible computational requirement can expose devices to denial of service attacks.
- We analyze the resilience of TurboBlom against node capture and show that TurboBlom can offer practically the same level of resilience against node capture as the Blom scheme.
- We propose a new secure mode for RPL using TurboBlom to enable efficient sender authentication. This new mode is particularly useful as many emerging applications in RPL networks rely on point-to-point communications (as opposed to merely relying on communications with the gateway).

The rest of the paper is organized as follows. Section 2 overviews related work. Section 3 covers the basics of the Blom Scheme. We introduce TurboBlom in Section 4, and analyze its resilience against node capture. We propose TAM in Section 5, and conclude the paper in Section 7.

2 Related work

Key predistribution

Early on, in the rise of large-volume industrial and consumer applications of wireless sensor networks, Carman, Kruus, and Matt [13] noticed that an adversary can physically capture nodes and acquire their data, since in practice they are typically left in an unattended/hostile environments and it is too costly, at large, to equip them with tamper-resistant hardware. In the quest for a solution, Eschenour and Gligor proposed the first random KPS in their seminal work [8]. Since then, many variants of this basic scheme have been proposed. These variants include schemes that use deterministic methods (usually based on combinatorial designs or error-correcting codes) in selecting subsets of keys for each node [14–20], and *multiple space* KPSs [21–23] which combine subset-based schemes with other key predistribution schemes such as the Blom scheme [10].

A notable variant of the Eschenour and Gligor (EG) scheme is the q -composite KPS due to Chan, Perrig and Song [9]. The q -composite scheme generalizes the EG scheme by requiring two devices to have at least q secret keys in common in order to establish a pairwise key. The q -composite offers greater resilience against node capture than EG when the number of nodes captured is relatively small, but is more vulnerable once a large number of nodes have been captured. This can be a desirable trade-off as small-scale attacks are less expensive and harder to detect [9].

When it comes to small-scale attacks, the optimum resilience is achieved by the Blom scheme [10]—it guarantees that the capture of a small number of nodes (up to the number of stored keys minus one) does not compromise any link between uncaptured nodes. In addition, the Blom scheme guarantees that any two nodes in the network are able to establish a pairwise key, whereas in the EG scheme and its variants, two nodes may not be able to establish a pairwise key as they may not have any secret key in common. The above advantages of the Blom scheme come at a price: to establish a pairwise key in the Blom scheme, a node must compute several field operations, including several computationally demanding field multiplications. In this work, we attempt to significantly reduce this cost by eliminating demanding field multiplications, while maintaining the scheme's high resilience against node capture. Our method is based on using random generator matrices such as random zero-one matrices in the Blom scheme.

RPL

Since RPL standardization in 2012, its security attracted a significant amount of research work on investigating potential threats, and proposing mitigation techniques for security attacks. In particular, many recent works have aimed at mitigating the sender's authentication problem (or closely related problems such as the Sybil attack) in RPL networks.

For instance, in a recent work, Raoof *et al.* [24] proposed a new secure mode, Chained Secure Mode (CSM), for the RPL protocol using intra-flow network coding. In CSM, each node encodes its outgoing messages using a randomly generated code, and updates its neighbors with the next code. While CSM can protect the network against external attackers, it is vulnerable to internal attacks. It is because CSM assumes that the first message comes from the

original sender. Therefore, an internal attacker can convince a node, say, u of being another node, say v , if the attacker sends the first message to u pretending to be v .

In another work, Airehrour *et al.* [25] proposed SecTrust-RPL, a new security framework for RPL. The authors proposed to install extra hardware on IoT devices to calculate a trust factor for each node. The trust factor is calculated based on successful packet transmissions between nodes. They showed that SecTrust-RPL can significantly mitigate the impact of Rank attack and Sybil attack. To protect the network against Sybil attack, the author proposed to bind the physical location of each node with its identity. Although this method can provide some degree of mitigation to the sender's authentication problem, a node can still send messages using another node's ID.

Despite many attempts in the literature (e.g., [26–29]) there is still no lightweight countermeasure that can prevent an adversary from spoofing, i.e. claiming the identity of a valid uncaptured node. In this work, however, we present a lightweight solution that prevents an adversary to use the identity of an uncaptured node unless the adversary extracts security keys of at least $\lambda + 1$ nodes, where λ is a security parameter.

3 Overview of the Blom scheme

In this section, we provide an overview of the Blom scheme. Table 1 provides the list of symbols used in this paper.

Bootstrapping phase

Consider a network with at most n nodes. In the “bootstrapping phase” of the Blom scheme, a trusted server generates two matrices: a public matrix G referred to as the *generator matrix*, and a private matrix D . The matrix G is a $(\lambda + 1) \times n$ matrix over F_q , where F_q denotes the finite field of order q , and λ is a security parameter. The matrix D , on the other hand, is a $(\lambda + 1) \times (\lambda + 1)$ symmetric matrix over F_q . The matrix D is a random matrix uniformly distributed over the set of all symmetric matrices over F_q .

After generating matrices D and G , the server computes

$$A = (DG)^T, \quad (1)$$

where $(DG)^T$ denotes the transpose of DG . It then preloads node u_i , $1 \leq i \leq n$, with the i th row of matrix A , denoted $A_{i,:}$, and the i th column of matrix G , denoted $G_{:,i}$.

Table 1. List of symbols.

Symbol	Description
$\lambda + 1$	No. of secret keys stored in each node
F_q	Finite field of order q
F_p	Finite field of order p , where p is a prime
G	The generator matrix
D	The private matrix
A	The matrix obtained from D and G as $A = (DG)^T$
$K_{i,j}$	Pairwise key between nodes i and j
$\mathcal{S}$	The set of elements of a confined generator matrix

<https://doi.org/10.1371/journal.pone.0295190.t001>

Key establishment phase

Let $K = AG$. Since D is symmetric, we get

$$K^T = (AG)^T = G^T A^T \stackrel{(1)}{=} G^T (DG) = G^T D^T G \stackrel{(1)}{=} AG = K, \quad (2)$$

thus K is a symmetric matrix, i.e., $K_{ij} = K_{ji}$. In the Blom scheme, K_{ij} (which is equal to K_{ji}) is used as the pairwise key between u_i and u_j .

To generate the pairwise key K_{ij} in a so-called *key establishment phase*, nodes u_i and u_j first exchange their public information, i.e., their columns of G . Then, as illustrated in Fig 1, they generate their pairwise key as

$$K_{ij} = \langle A_{i,:}, G_{:,j} \rangle = \langle A_{j,:}, G_{:,i} \rangle \quad (3)$$

where $\langle x, y \rangle$ denotes the inner product of vectors x and y .

Resilience of the Blom scheme

If every $\lambda + 1$ columns of the generator matrix G are linearly independent, it can be proven that no information about the pairwise keys between uncaptured nodes is revealed if the number of captured nodes is at most λ [10, 22]. We will generalize this result later in Theorem 2.

Constructing matrix G

When $q > n$, we can construct a matrix G in which every $\lambda + 1$ columns are linearly independent. For instance, it can be shown that when $q > n$, every $\lambda + 1$ columns of the following

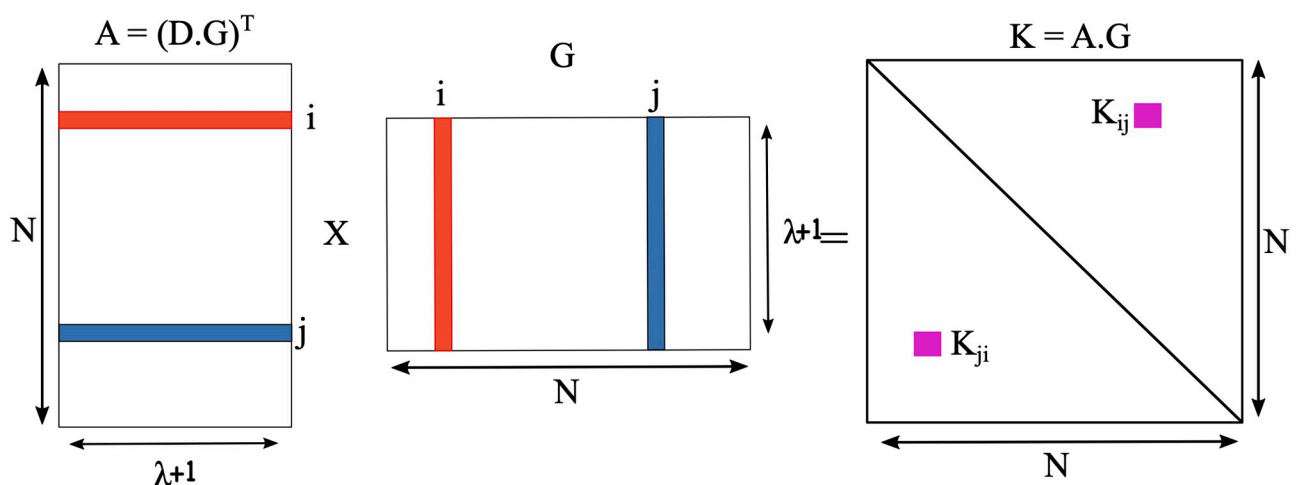


Fig 1. The pairwise key between nodes u_i and u_j is the element K_{ij} of matrix K , where $K = A \cdot G$.

<https://doi.org/10.1371/journal.pone.0295190.g001>

Vandermonde matrix (with $g \in F_q$ being a primitive element) are linearly independent [22].

$$G = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ g & g^2 & g^3 & \cdots & g^n \\ g^2 & (g^2)^2 & (g^3)^2 & \cdots & (g^n)^2 \\ & & & \ddots & \\ g^\lambda & (g^2)^\lambda & (g^3)^\lambda & \cdots & (g^n)^\lambda \end{bmatrix} \quad (4)$$

An advantage of using a Vandermonde matrix as the generator matrix in the Blom scheme is that node u_i only needs to store g^i , instead of the entire i th column of G (i.e. $G_{:,i}$), since the entire column $G_{:,i}$ can be constructed from g^i —to construct the i th column of G , however, the node needs to perform $\lambda - 1$ field multiplications to compute $(g^i)^2, (g^i)^3, \dots, (g^i)^\lambda$ from g^i .

Example 1. Consider a network with $n = 5$ nodes. Let $q = 19, \lambda = 2$, and

$$G = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 2 & 4 & 8 & 16 & 13 \\ 4 & 16 & 7 & 9 & 17 \end{bmatrix} \quad D = \begin{bmatrix} 6 & 15 & 14 \\ 15 & 1 & 8 \\ 14 & 8 & 3 \end{bmatrix}$$

Notice that Matrix D is symmetric, and every four columns of matrix G are linearly independent. The matrix A is then calculated as:

$$A = (DG)^T = \begin{bmatrix} 16 & 11 & 4 \\ 5 & 14 & 18 \\ 15 & 3 & 4 \\ 11 & 8 & 17 \\ 2 & 12 & 17 \end{bmatrix}$$

Before being deployed in the network, node u_i is preloaded with the i th column of G and the i th row of A as its public and private information, respectively. Assume nodes u_2 and u_5 want to communicate securely with each other after they are deployed in the network. To establish a pairwise key, node u_2 calculates the following:

$$K_{2,5} = \langle A_{2,:}, G_{:,5} \rangle = [5 \quad 14 \quad 18] \cdot \begin{bmatrix} 1 \\ 13 \\ 17 \end{bmatrix} = 18$$

Similarly, node u_5 computes $K_{5,2}$ as

$$K_{5,2} = \langle A_{5,:}, G_{:,2} \rangle = [2 \quad 12 \quad 17] \cdot \begin{bmatrix} 1 \\ 4 \\ 16 \end{bmatrix} = 18$$

In this example, the secret pairwise key between nodes u_2 and u_5 is 18. Note that, to provide enough security, the order of finite fields used in practice will be much larger than 19.

4 TurboBlom

In this section, we introduce TurboBlom, and show how it significantly reduces the computational cost of the Blom scheme while maintaining a high resilience against node capture.

Recall that nodes u_i and u_j need to compute the inner product (3) to generate a pairwise secret key. Computing this inner product requires $\lambda + 1$ field multiplications and λ field additions. Computing a field multiplication, in general, is considerably harder than computing a field addition. For example, Figs 2 and 3 show the average number of CPU clock cycles needed to compute a field multiplication and a field addition, respectively, on 16-bit MSP430 micro-processor family—this is a family of processors commonly used in IoT devices such as Tmote Sky and Zolertia Z1 [30, 31]. As shown in these figures, computing a field multiplication over a field of order 256 bits, for instance, requires 547 times more clock cycles than computing a field addition. Since computing field multiplications is the main cost of generating a pairwise key in the Blom scheme, in TurboBlom we aim to eliminate field multiplications in order to speed up pairwise key generation.

Eliminating field multiplications

As mentioned earlier, a node must compute the inner product (3) to generate a pairwise key. We can eliminate the need for field multiplication in (3) if we use a “confined” generator matrix G . In a confined matrix, elements can only come from a subset $\mathcal{S} \subset F_q$. For example, if we set $\mathcal{S} = \{0, 1\}$, we limit the elements of the generator matrix G to 0 and 1. That is, each element of G can be either the number 0 or the number 1, where 0 and 1 represent, respectively, the additive and multiplicative identities of F_q .

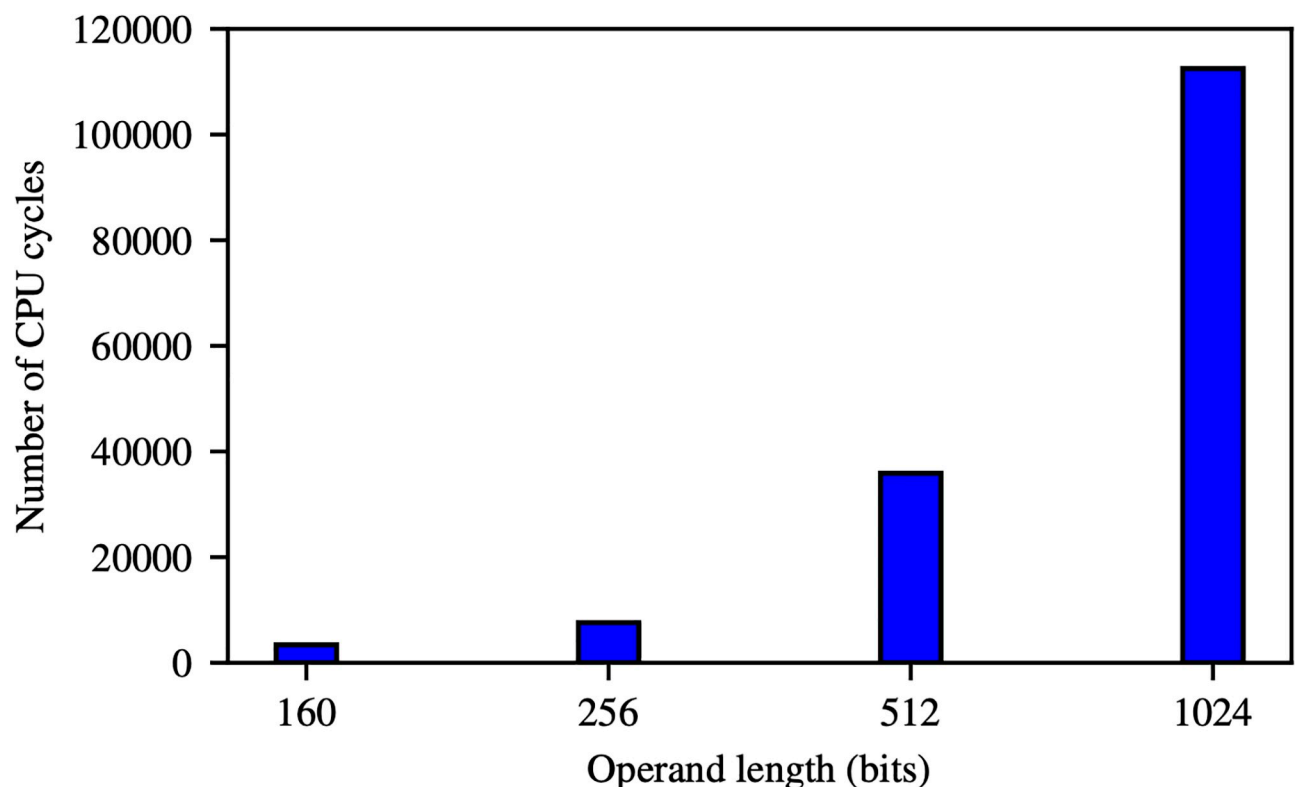


Fig 2. The average number of CPU cycles needed to compute a single field multiplication in MSP430 processor.

<https://doi.org/10.1371/journal.pone.0295190.g002>

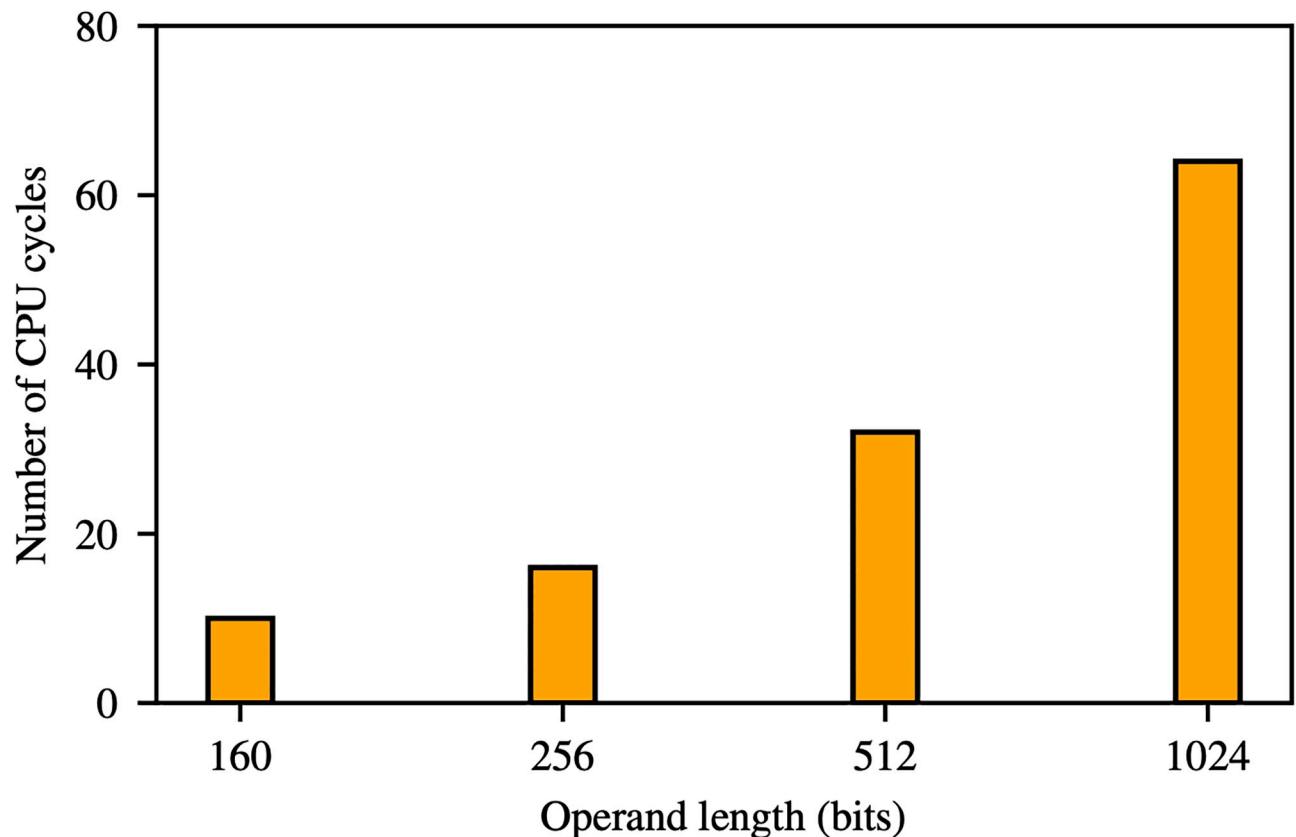


Fig 3. The average number of CPU cycles needed to compute a single field addition in MSP430 processor.

<https://doi.org/10.1371/journal.pone.0295190.g003>

Such a “zero-one matrix” (We refer to the matrix as zero-one matrix rather than binary matrix to stress that the matrix is over F_q rather than F_2 , although its elements are limited to zero and one) eliminates the need for field multiplication, because it turns every multiplications in (3) into a multiplication by 0 or a multiplication by 1 (which are both trivial). This makes pairwise key generation in the Blom scheme significantly faster. The important concern with using a confined generator matrix, such as a zero-one generator matrix, is its potential negative impact on the resilience of the scheme. A major contribution of this work is to answer this concern. Looking ahead, we later in this section show that this impact can be made exponentially small in practice.

Remark 1. Although the elements of G are limited to a small subset of F_q , all the operations in computing the inner product (3) are still performed in the field F_q . Moreover, the pairwise key between nodes can still take any value from F_q , because the elements of matrix D can take any value from F_q (see Example 2). We will formally prove this in Theorem 2.

We can also limit the elements of matrix G to

$$\mathcal{S} = \{-1, 0, 1\}.$$

This, too, eliminates field multiplications, because multiplication by -1 is a fast operation (recall that $-1 \neq 1$ in any finite field of odd order). This idea can be extended to even larger sets $\mathcal{S}$ if we use a finite field F_p in which p is a Mersenne prime (in general, it is possible to use a field with characteristic p , where p is a Mersenne prime). A Mersenne prime is a prime of the form $2^n - 1$, where n is an integer. An interesting property of F_p , with p being a Mersenne

prime, is that multiplying a number $a \in F_p$ by a power of two (i.e., a number of the form 2^i) is simply a circular shift of a , when a is represented in binary. In addition, negating a number a (i.e., computing $-a$) can be performed easily by reversing all the bits of a , when a is represented in binary. Considering the above properties, we can set

$$\mathcal{S} = \{-2^{n-1}, \dots, -2^1, -2^0, 0, 2^0, 2^1, \dots, 2^{n-1}\}. \quad (5)$$

Since all the non-zero elements of the above set $\mathcal{S}$ are powers of two, every field multiplication is either eliminated (due to multiplication by zero), or replaced by a simple shift operation (due to multiplication by a power of two), possibly followed by bit reversal (due to negation).

Remark 2. The set $\mathcal{S}$ defined in (5) has $2n + 1$ distinct elements, i.e. $|\mathcal{S}| = 2n + 1$. This is because $(2^i \neq 2^j \bmod p)$ and $(2^i \neq -2^j \bmod p)$ for any $0 \leq i \neq j \leq n - 1$, where $p = 2^n - 1$.

Example 2. Consider a network with $n = 4$ nodes, u_1, u_2, u_3 and u_4 . Let $\lambda = 1$, and

$$G = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & -1 \end{bmatrix} \quad D = \begin{bmatrix} a & c \\ c & b \end{bmatrix}$$

where the elements of the secret matrix D (i.e., a, b , and c) are random numbers uniformly distributed over F_q , and q is an odd number (hence $-1 \neq 1$). Notice that Matrix D is symmetric, and every two columns of G are linearly independent. The matrix A is then calculated as:

$$A = (DG)^T = \begin{bmatrix} c & b \\ a & c \\ a + c & c + b \\ a - c & c - b \end{bmatrix}$$

Before being deployed in the network, node u_i is preloaded with the i th row of A . For instance, u_1 is preloaded with secret numbers c and b , and u_2 is preloaded with secret numbers a and c . Fig 4 shows the set of two secret numbers each node is preloaded with, as well as the pairwise key between each pair of nodes (the number on each edge). For instance, as shown in the figure, the pairwise key between u_2 and u_3 is the number $a + c$. Note that, although the elements of matrix G are limited to $\mathcal{S} = \{-1, 0, 1\}$, the pairwise key between two nodes can take any number from F_q . This is because the elements of D (i.e., a, b , and c) are uniformly distributed over F_q . Also, notice that each node can compute a pairwise key with any other node using its preloaded numbers. For instance, u_4 can compute the pairwise key $a - b$ with node u_3 by adding its two secret numbers, i.e. $(a - c) + (c - b)$.

As illustrated in Fig 5, suppose that the adversary captures node u_1 and extracts the secret numbers c and b . This will clearly compromise the links between node u_1 and the remaining three nodes u_2, u_3 and u_4 . The adversary, however, gains no information about pairwise keys between uncaptured nodes (i.e., pairwise keys corresponding to green solid links) because the adversary has no information about a (as a is uniformly distributed over F_q). Similarly, we can verify that if adversary captures another node, they do not gain any information about the pairwise key between uncaptured nodes.

Remark 3. In Example 2, the elements of matrix G were limited to the set $\mathcal{S} = \{-1, 0, 1\}$. Yet, the pairwise keys between two nodes are uniformly distributed over F_q . Furthermore, the adversary gains no information about the pairwise key of two nodes by capturing another node in the network.

Speedup of TurboBlom

In this section, we compare the speed of the TurboBlom and Blom schemes. In the next section, we will compare the resilience of the two schemes, a considerably more challenging task.

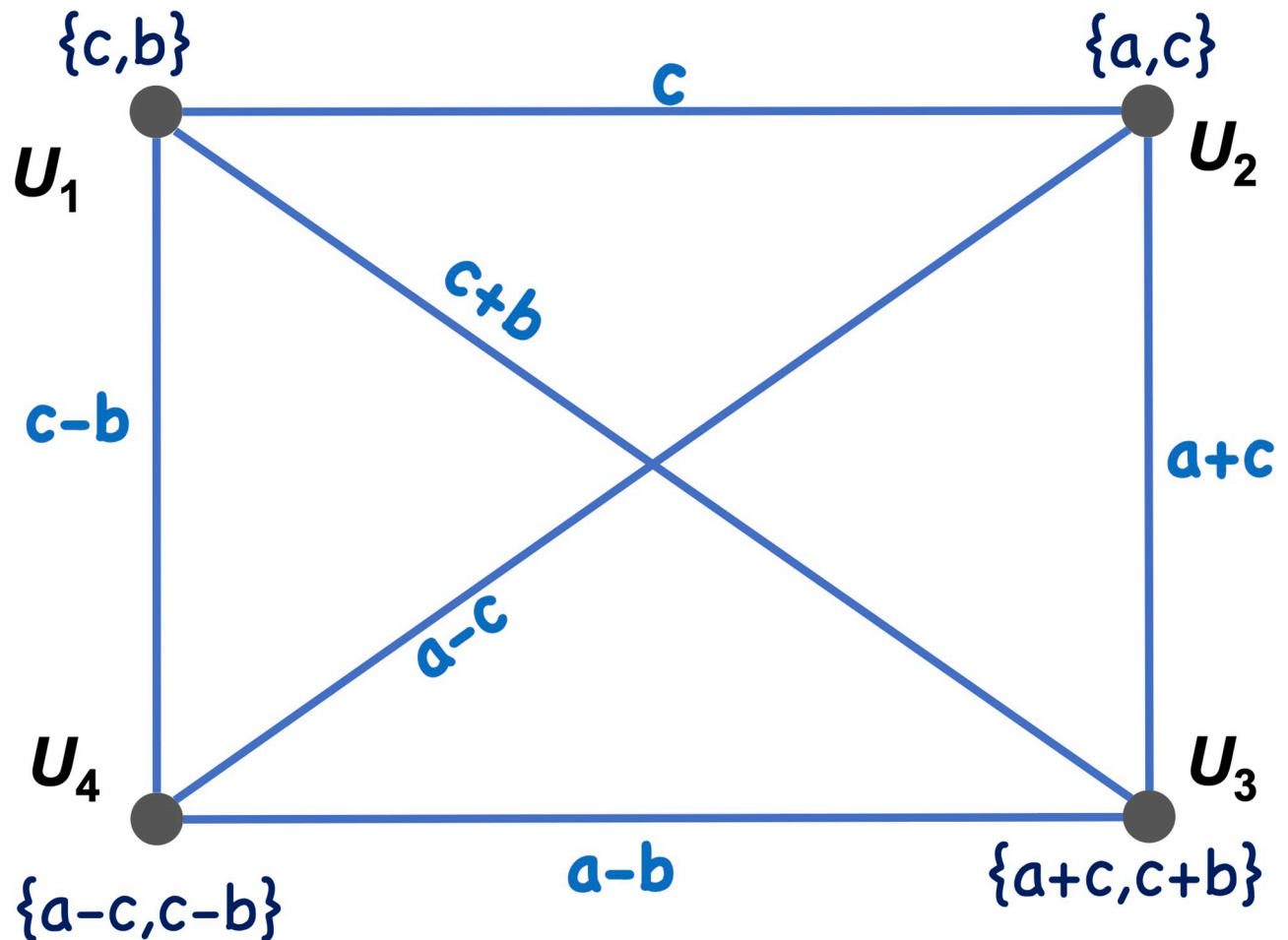


Fig 4. Each node u_1 , u_2 , u_3 , and u_4 is preloaded with a pair of numbers. The number on each edge is the pairwise key between the two nodes incident to the edge.

<https://doi.org/10.1371/journal.pone.0295190.g004>

To compare the speed of the two schemes, we can compare the number of operations that each scheme needs. This is not difficult as both schemes perform the same inner product (3) to generate pairwise key. In the Blom scheme, computing this inner product needs $\lambda + 1$ field multiplications and λ field additions. In the TurboBlom scheme, however, no field multiplication is required: any field multiplication in the Blom scheme is either completely eliminated in the TurboBlom scheme or is replaced with a simple operation such as circular shift, which is essentially as fast as field addition. Using this insight, the following theorem evaluates the speedup of TurboBlom over the Blom scheme.

Theorem 1. Suppose field multiplication and addition require, respectively, m and a units of time.

Then, the TurboBlom's speedup over the Blom scheme is at least

$$\frac{1}{2} \cdot \left(\frac{m}{a} + 1 \right).$$

Moreover, if we use a zero-one matrix in TurboBlom, the above speedup is improved to at least $\left(\frac{m}{a} + 1 \right)$, in the worst case, and to at least $2 \cdot \left(\frac{m}{a} + 1 \right)$, in the average.

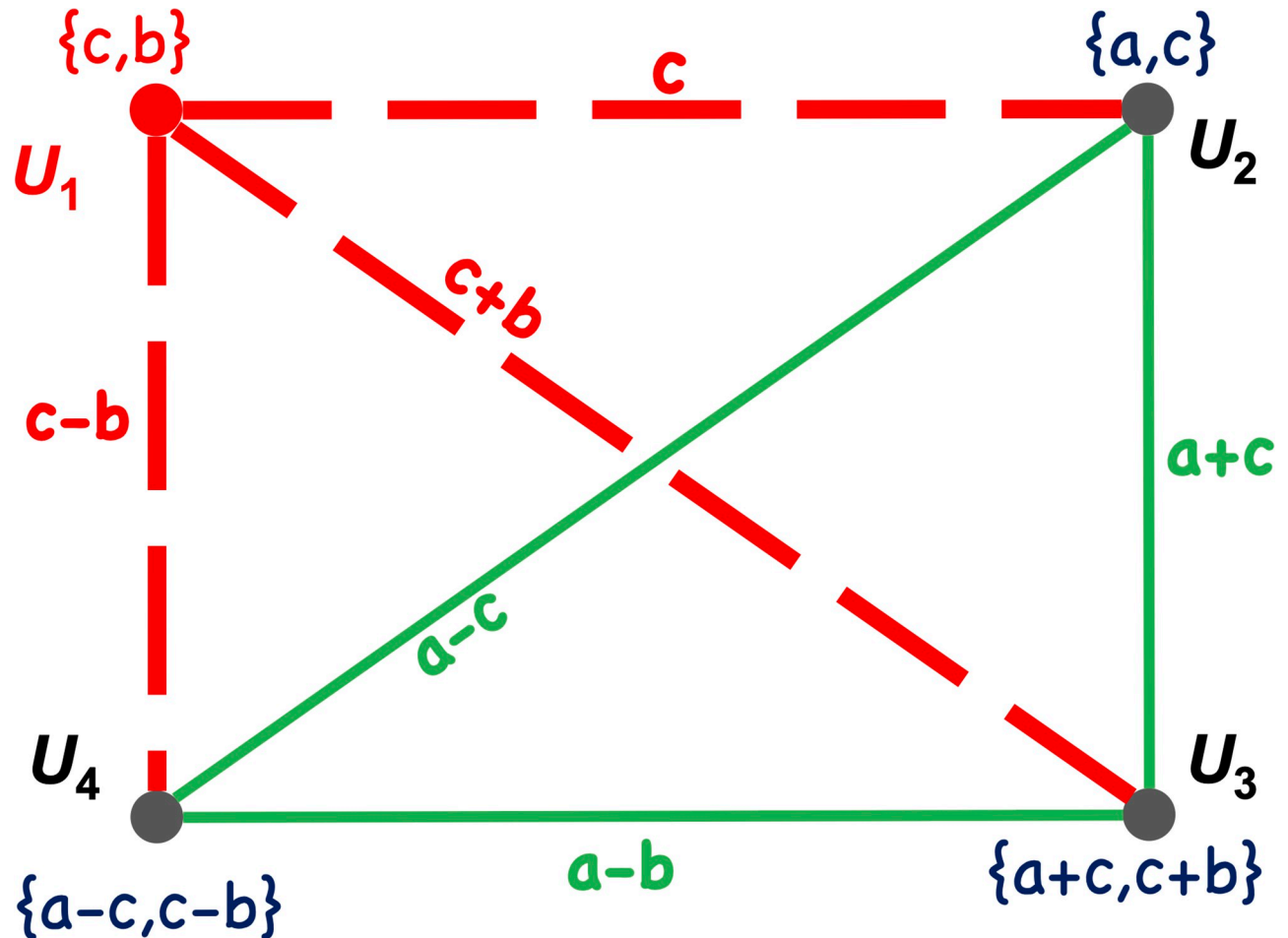


Fig 5. The adversary captures node u_1 and extracts the secret numbers c and b . This will compromise the links between u_1 and the other three nodes (the red dashed links). The adversary, however, gains no information about the pairwise keys between u_2, u_3 and u_4 , which are $a - c$, $a - b$ and $a + c$. This is because a is uniformly distributed over F_q , hence $a - c$, $a - b$ and $a + c$ are all uniformly distributed over F_q .

<https://doi.org/10.1371/journal.pone.0295190.g005>

Proof. To generate a pairwise key specified by (3), the Blom scheme requires $\lambda + 1$ field multiplications and λ field additions. This translates to the computation time of

$$(\lambda + 1) \cdot m + \lambda \cdot a$$

In TurboBlom, each field multiplication is either eliminated or replaced with a simple operation (circular shift or bit reverse) which is practically as fast as field addition. Therefore, in the worst case, TurboBlom needs $2\lambda + 1$ field additions, which translates to the computation time of

$$(\lambda + 1) \cdot a + \lambda \cdot a.$$

Therefore, we get the speedup of at least

$$\frac{(\lambda + 1) \cdot m + \lambda \cdot a}{(\lambda + 1) \cdot a + \lambda \cdot a} \geq \frac{1}{2} \cdot \left(\frac{m}{a} + 1 \right).$$

If TurboBlom uses a zero-one matrix, then all of the field multiplications are eliminated. Therefore, in this case the speedup will be

$$\frac{(\lambda + 1) \cdot m + \lambda \cdot a}{0 \cdot m + \lambda \cdot a} \geq \frac{m}{a} + 1.$$

We note that, on average, half of all of the field additions are eliminated (due to zeros in the generator matrix) when the elements of the zero-one generator matrix are selected uniformly at random. Let $1 \leq \lambda' \leq \lambda$ be a random variable denoting the number of required field additions. Then, the expected speedup can be calculated as

$$\begin{aligned} \mathbb{E} \left[\frac{(\lambda + 1) \cdot m + \lambda \cdot a}{0 \cdot m + \lambda' \cdot a} \right] &= \mathbb{E} \left[\frac{(\lambda + 1) \cdot m + \lambda \cdot a}{\lambda' \cdot a} \right] \\ &= \left(\frac{(\lambda + 1) \cdot m + \lambda \cdot a}{a} \right) \cdot \mathbb{E} \left[\frac{1}{\lambda'} \right] \\ &\geq \left(\frac{m}{a} + 1 \right) \cdot \lambda \cdot \mathbb{E} \left[\frac{1}{\lambda'} \right] \\ &\geq \left(\frac{m}{a} + 1 \right) \cdot \lambda \cdot \frac{1}{\mathbb{E}[\lambda']} \\ &= \left(\frac{m}{a} + 1 \right) \cdot \lambda \cdot \frac{1}{\left(\frac{\lambda}{2} \right)} \\ &= 2 \cdot \left(\frac{m}{a} + 1 \right), \end{aligned}$$

Where the last inequality is by the fact that

$$\mathbb{E} \left[\frac{1}{x} \right] \geq \frac{1}{\mathbb{E}[x]}$$

for any positive random variable x . Note that Jensen's inequality states that $\mathbb{E}[\phi(x)] \geq \phi(\mathbb{E}[x])$ for any convex function ϕ . The function $\phi(x) = \frac{1}{x}$ is convex when $x > 0$, hence the inequality.

Example 3. The speedup of TurboBlom can be significant in practice. For instance, as illustrated in Figs 2 and 3, when the field size is 256 bits, we have $\frac{m}{a} \approx 547$. Therefore, if we use a zero-one matrix in TurboBlom, we can expect a speedup of around $2(547 + 1) = 1,096$ over the Blom scheme (i.e., TurboBlom requires about three orders of magnitude less number clock cycles as the Blom scheme to compute a pairwise key).

Resilience of TurboBlom

As mentioned earlier, we can eliminate all the field multiplications in the pairwise key generation process by using a confined generator matrix G . This makes TurboBlom significantly lighter and faster than the original Blom scheme. In addition, as we will show in the remaining of this section, TurboBlom can maintain a high resilience against node capture. To this end, we present Theorem 2, which proves that an adversary who has captured a number of nodes and extracted their secret keys gains no information about the pairwise key between any two uncaptured nodes u_i and u_j if neither the i th column nor the j th column of G is a linear combination of the columns of G that correspond to the captured nodes. Notably, this result holds for any generator matrix G including zero-one matrices.

Prior to presenting Theorem 2, let us first formally define what we mean by “adversary gains no information”.

Definition 1 (Adversary gaining no information). Let us fix the generator matrix G to an arbitrary $(\lambda + 1) \times n$ matrix over F_q . Suppose the adversary has captured and extracted secret keys of nodes u_i , $i \in \mathcal{L}$, where $\mathcal{L}$ represents the set of captured nodes. Let the $1 \times (\lambda + 1)$ vector A_i over F_q denote the secret keys of node u_i , $i \in \mathcal{L}$. Let $\mathcal{D}$ be the set of all $(\lambda + 1) \times (\lambda + 1)$ symmetric matrices that, when used with the generator matrix G , result in A_i for every $i \in \mathcal{L}$, that is

$$\mathcal{D} = \{D | \forall i \in \mathcal{L} : (D \cdot G_{:,i})^T = A_i\}$$

Given the above setup, consider a probabilistic experiment in which the random variable D is uniformly distributed over $\mathcal{D}$. Let $K_{s,d}$ be the random variable representing the pairwise key between nodes u_s and u_d . We say that the adversary gains no information about $K_{s,d}$ if and only if $K_{s,d}$ is uniformly distributed over F_q .

Lemma 1. Let $\mathcal{V}$ be a subspace of the vector space F_q^m , where m is a positive integer. Let $\mathcal{W}$ denote the null space of $\mathcal{V}$.

Then, for every vectors $\mathbf{y}_1, \mathbf{y}_2 \in F_q^m \setminus \mathcal{V}$, there exists a vector $\mathbf{w} \in \mathcal{W}$ such that $\mathbf{w}$ is not orthogonal to either $\mathbf{y}_1$ or $\mathbf{y}_2$, i.e.

$$\forall \mathbf{y}_1, \mathbf{y}_2 \in F_q^m \setminus \mathcal{V}, \exists \mathbf{w} \in \mathcal{W} : \langle \mathbf{y}_1, \mathbf{w} \rangle \neq 0 \wedge \langle \mathbf{y}_2, \mathbf{w} \rangle \neq 0.$$

Proof. A vector $\mathbf{v} \in F_q^m$ is orthogonal to every vector in $\mathcal{W}$ if and only if $\mathbf{v} \in \mathcal{V}$. Since $\mathbf{y}_1 \notin \mathcal{V}$, then there must exist a vector in $\mathcal{W}$ which is not orthogonal to $\mathbf{y}_1$. Therefore, the set $\mathcal{W}_1$ defined as

$$\mathcal{W}_1 = \{\mathbf{w} \in \mathcal{W} | \langle \mathbf{y}_1, \mathbf{w} \rangle \neq 0\}$$

is non-empty. Similarly, the set $\mathcal{W}_2$

$$\mathcal{W}_2 = \{\mathbf{w} \in \mathcal{W} | \langle \mathbf{y}_2, \mathbf{w} \rangle \neq 0\}$$

must be non-empty. If $(\mathcal{W}_1 \cap \mathcal{W}_2) \neq \emptyset$, then we are done because any vector

$$\mathbf{w} \in \mathcal{W}_1 \cap \mathcal{W}_2$$

is not orthogonal to either $\mathbf{y}_1$ or $\mathbf{y}_2$. Otherwise, we can set $\mathbf{w} = \mathbf{w}_1 + \mathbf{w}_2$, where $\mathbf{w}_1 \in \mathcal{W}_1$ and $\mathbf{w}_2 \in \mathcal{W}_2$. Note that $\mathbf{w} \in \mathcal{W}$. Furthermore, $\mathbf{w}_1 \notin \mathcal{W}_2$ and $\mathbf{w}_2 \notin \mathcal{W}_1$ because $(\mathcal{W}_1 \cap \mathcal{W}_2) = \emptyset$. Therefore, by the definition of sets $\mathcal{W}_1$ and $\mathcal{W}_2$, we get

$$\langle \mathbf{y}_1, \mathbf{w}_2 \rangle = 0 \tag{6}$$

and

$$\langle \mathbf{y}_2, \mathbf{w}_1 \rangle = 0. \tag{7}$$

Thus

$$\begin{aligned} \langle \mathbf{y}_1, \mathbf{w} \rangle &= \langle \mathbf{y}_1, (\mathbf{w}_1 + \mathbf{w}_2) \rangle \\ &= \langle \mathbf{y}_1, \mathbf{w}_1 \rangle + \langle \mathbf{y}_1, \mathbf{w}_2 \rangle \\ &\stackrel{(6)}{=} \langle \mathbf{y}_1, \mathbf{w}_1 \rangle + 0 \\ &\neq 0. \end{aligned}$$

and

$$\begin{aligned}
 \langle \mathbf{y}_2, \mathbf{w} \rangle &= \langle \mathbf{y}_2, (\mathbf{w}_1 + \mathbf{w}_2) \rangle \\
 &= \langle \mathbf{y}_2, \mathbf{w}_1 \rangle + \langle \mathbf{y}_2, \mathbf{w}_2 \rangle \\
 &\stackrel{(\dagger)}{=} 0 + \langle \mathbf{y}_2, \mathbf{w}_2 \rangle \\
 &\neq 0.
 \end{aligned}$$

Thus, the vector $\mathbf{w} = \mathbf{w}_1 + \mathbf{w}_2$ is not orthogonal to either $\mathbf{y}_1$ or $\mathbf{y}_2$.

Lemma 2. Let B be a non-zero matrix over F_q and $\mathcal{A}$ be a non-empty set of matrices over F_q such that

$$\forall c \in F_q : A \in \mathcal{A} \Rightarrow (A + c \cdot B) \in \mathcal{A}.$$

Then, the binary relation $\sim$ on $\mathcal{A}$ defined as

$$\forall A_1, A_2 \in \mathcal{A} : A_1 \sim A_2 \Leftrightarrow A_1 - A_2 = c \cdot B \text{ for some } c \in F_q$$

is an equivalence relation, which partitions $\mathcal{A}$ into equivalent classes $[A_i]$ of size q , where $A_i \in \mathcal{A}$ and

$$[A_i] = \{A | A = A_i + c \cdot B, c \in F_q\}.$$

Proof. The binary relation $\sim$ on $\mathcal{A}$ is an equivalence relation because it is reflexive, symmetric and transitive:

- (Reflexivity) $\forall A \in \mathcal{A} : A \sim A$, because $A - A = 0 \cdot B, c = 0 \in F_q$.
- (Symmetry) $\forall A_1, A_2 \in \mathcal{A} : (A_1 \sim A_2) \Rightarrow (A_2 \sim A_1)$, because

$$\begin{aligned}
 (A_1 \sim A_2) &\Rightarrow A_1 - A_2 = c \cdot B, c \in F_q \\
 &\Rightarrow A_2 - A_1 = (-c) \cdot B, -c \in F_q \\
 &\Rightarrow (A_2 \sim A_1).
 \end{aligned}$$

- (Transitivity) $\forall A_1, A_2, A_3 \in \mathcal{A} :$

$$(A_1 \sim A_2) \wedge (A_2 \sim A_3) \Rightarrow (A_1 \sim A_3)$$

because

$$\begin{aligned}
 &(A_1 \sim A_2) \wedge (A_2 \sim A_3) \\
 &\Rightarrow (A_1 - A_2 = c_1 \cdot B) \wedge (A_2 - A_3 = c_2 \cdot B), c_1, c_2 \in F_q \\
 &\Rightarrow A_1 - A_3 = (c_1 + c_2) \cdot B, (c_1 + c_2) \in F_q \\
 &\Rightarrow (A_1 \sim A_3).
 \end{aligned}$$

Therefore, the binary relation $\sim$ partitions $\mathcal{A}$ into disjoint equivalence classes

$$\begin{aligned}
 [A_i] &= \{A | A \sim A_i\} \\
 &= \{A | A = A_i + c \cdot B, c \in F_q\}.
 \end{aligned}$$

Finally, note that the coefficient $c \in F_q$ takes q distinct values. Furthermore, B is non-zero, thus

$$(A_i + c_1 \cdot B = A_i + c_2 \cdot B) \Leftrightarrow c_1 = c_2.$$

Therefore, every class $[A_i]$ has exactly q distinct elements.

Theorem 2. Let G be an arbitrary $(\lambda + 1) \times n$ matrix over F_q , where λ and n are positive integers. Note that, as a special case, G can be a zero-one matrix. Suppose we use G as the generator matrix of the Blom scheme. Suppose the secret random matrix D is uniformly distributed over the set of all $(\lambda + 1) \times (\lambda + 1)$ symmetric matrices. Assume that an adversary has captured and extracted the secret keys of a set $\mathcal{L}$ of $l \geq 1$ nodes. Let $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$ be the column vectors of G that correspond to these l captured nodes. Let u_s and u_d be any two uncaptured nodes, and $\mathbf{g}_s$ and $\mathbf{g}_d$ be the two column vectors of G that correspond to u_s and u_d , respectively.

Then, the adversary gains no information about the pairwise key between u_s and u_d if and only if neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$.

Proof. Let $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_l$ be the row vectors of A that correspond to the captured nodes. We have

$$\forall 1 \leq i \leq l: \mathbf{a}_i = (D\mathbf{g}_i)^T, \quad (8)$$

because $A = (DG)^T$.

Suppose $\mathbf{g}_s$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$, that is

$$\mathbf{g}_s = \sum_{i=1}^l c_i \cdot \mathbf{g}_i, \quad (9)$$

for some $c_i \in F_q$. The adversary knows the values of c_i , $1 \leq i \leq l$, because the generator matrix G is public. In addition, the adversary knows $\mathbf{a}_i$, $1 \leq i \leq l$, which are the secret keys of the captured nodes. Therefore, by (8) and (9), the adversary can compute $\mathbf{a}_s$ as

$$\mathbf{a}_s = \sum_{i=1}^l c_i \cdot \mathbf{a}_i, \quad (10)$$

because

$$\begin{aligned} \mathbf{a}_s &= (D\mathbf{g}_s)^T \\ &\stackrel{(9)}{=} \left(D \cdot \sum_{i=1}^l c_i \cdot \mathbf{g}_i \right)^T \\ &= \left(\sum_{i=1}^l c_i \cdot D\mathbf{g}_i \right)^T \\ &= \sum_{i=1}^l c_i \cdot (D\mathbf{g}_i)^T \\ &\stackrel{(8)}{=} \sum_{i=1}^l c_i \cdot \mathbf{a}_i. \end{aligned}$$

Therefore, if $\mathbf{g}_s$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$, the adversary can compute the secret keys of node u_s (i.e., $\mathbf{a}_s$), thus can compute the pairwise key between u_s and any other node (including u_d). Similarly, if $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$, the adversary can compute the secret keys of node u_d .

Next, we prove the challenging part, which is that the adversary gains no information about the pairwise key between u_s and u_d if neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$. To this end, we assume that neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$.

Let G_0 be the submatrix of G that includes only the column vectors $\mathbf{g}_i$, $1 \leq i \leq l$. Let A_0 be the submatrix of A that includes only the row vectors $\mathbf{a}_i$, $1 \leq i \leq l$. By (1), we have

$$A_0 = (DG_0)^T$$

or equivalently

$$A_0^T = DG_0 \quad (11)$$

As in Definition 1, we define $\mathcal{D}$ to be the set of all symmetric matrices D that satisfy Eq (11). Note that the set $\mathcal{D}$ is not empty, because the secret matrix D used in the Blom scheme satisfies (11).

Let $\mathcal{V}$ denote the column space of G_0 , and $\mathcal{W}$ be the null space of $\mathcal{V}$. Recall that neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$. Thus, $\mathbf{g}_s, \mathbf{g}_d \notin \mathcal{V}$. Therefore, by Lemma 1, there exists $\mathbf{w} \in \mathcal{W}$ such that $\mathbf{w}$ is not orthogonal to either $\mathbf{g}_s$ or $\mathbf{g}_d$. Let

$$\mathbf{w} = (w_1, w_2, \dots, w_{\lambda+1})$$

and B be a $(\lambda + 1) \times (\lambda + 1)$ matrix over F_q whose i th row, $1 \leq i \leq \lambda + 1$, is $w_i \cdot \mathbf{w}$. The matrix B is symmetric because

$$B_{ij} = B_{ji} = w_i \cdot w_j.$$

Furthermore, we have

$$BG_0 = \mathbf{0},$$

because each row of B is in $\mathcal{W}$, hence is orthogonal to every column of G_0 . Therefore, we get

$$\forall c \in F_q : D \in \mathcal{D} \Rightarrow (D + c \cdot B) \in \mathcal{D},$$

because

$$\begin{aligned} (D + c \cdot B)G_0 &= DG_0 + c \cdot BG_0 \\ &= A_0^T \end{aligned}$$

that is $(D + c \cdot B)$ satisfies (11), hence is in $\mathcal{D}$ by definition. Let us define the binary relation $\sim$ on $\mathcal{D}$ as

$$\forall D_1, D_2 \in \mathcal{D} : D_1 \sim D_2 \Leftrightarrow D_1 - D_2 = c \cdot B \text{ for some } c \in F_q.$$

By Lemma 2, the binary relation $\sim$ is an equivalence relation which partitions $\mathcal{D}$ into equivalent classes of size q . Let $[D_i]$ denote the i th, $1 \leq i \leq k$, equivalent class represented by the matrix $D_i \in \mathcal{D}$. Since the matrix D in the Blom scheme is uniformly distributed over $\mathcal{D}$, it can be represented as

$$D = D_j + r \cdot B, \quad (12)$$

where j and r are random variables uniformly distributed over $\{1, 2, \dots, k\}$, and F_q , respectively. The random variable j indicates the class to which matrix D belongs, and the random variable r indicates which element of the class $[D_j]$ is the matrix D .

Let

$$a_s = \langle \mathbf{g}_s, \mathbf{w} \rangle, \quad (13)$$

and

$$a_d = \langle \mathbf{g}_d, \mathbf{w} \rangle, \quad (14)$$

Note that $a_s \neq 0$ and $a_d \neq 0$, because $\mathbf{w}$ is not orthogonal to either $\mathbf{g}_s$ or $\mathbf{g}_d$. By (1) and (3), the pairwise key between u_s and u_d is

$$\begin{aligned} K_{s,d} &\stackrel{(3)}{=} \mathbf{a}_s \cdot \mathbf{g}_d \\ &\stackrel{(1)}{=} (\mathbf{g}_s^T \cdot D) \cdot \mathbf{g}_d \\ &\stackrel{(12)}{=} (\mathbf{g}_s^T \cdot (D_j + r \cdot B)) \cdot \mathbf{g}_d \\ &= (\mathbf{g}_s^T \cdot D_j + r \cdot (\mathbf{g}_s^T \cdot B)) \cdot \mathbf{g}_d \\ &= (\mathbf{g}_s^T \cdot D_j + r \cdot (B \cdot \mathbf{g}_s)^T) \cdot \mathbf{g}_d \\ &\stackrel{(13)}{=} (\mathbf{g}_s^T \cdot D_j + r \cdot (a_s \cdot \mathbf{w})^T) \cdot \mathbf{g}_d \\ &= (\mathbf{g}_s^T \cdot D_j + r \cdot (a_s \cdot \mathbf{w}^T)) \cdot \mathbf{g}_d \\ &= (\mathbf{g}_s^T \cdot D_j) \cdot \mathbf{g}_d + r \cdot a_s \cdot (\mathbf{w}^T \cdot \mathbf{g}_d) \\ &\stackrel{(14)}{=} (\mathbf{g}_s^T \cdot D_j) \cdot \mathbf{g}_d + \underbrace{r \cdot a_s \cdot a_d}_R \end{aligned}$$

Recall that a_s and a_d are non-zero elements, and r is distributed uniformly over F_q . Therefore, $R = r \cdot a_s \cdot a_d$, and consequently $K_{s,d}$ is distributed uniformly over F_q . Therefore, by Definition 1, the adversary gains no information about $K_{s,d}$. This concludes the proof. Using Theorem 2, we next evaluate the resilience of TurboBlom. We start with defining $\text{fail}(l)$, a common measure of resilience against node capture.

Definition 2 (Resilience: $\text{fail}(l)$). We measure resilience of a scheme by the extent to which the scheme can withstand an adversary that captures a number of nodes. In line with previous works (e.g., [23]), we quantify this measure by $\text{fail}(l)$, which is defined as the probability that the adversary gains information (as per Definition 1) about the pairwise key between two random nodes u_i and u_j by capturing l randomly chosen nodes not involving u_i and u_j . The probability space is determined by the above random choices, as well as the random choices made in constructing the secret matrix D , and the generator matrix G .

In the remaining of this section, we analyze $\text{fail}(l)$ for TurboBlom. In Theorems 3, 4, and 5, we prove that $\text{fail}(l)$ can be made exponentially small in TurboBlom when the generator matrix G is a random matrix.

Theorem 3. Let λ and n be positive integers, $\mathcal{S}$ be a non-empty subset of F_q , $\mathcal{G}$ be the set of all $(\lambda + 1) \times n$ matrices over $\mathcal{S}$, and $\mathcal{D}$ be the set of all $(\lambda + 1) \times (\lambda + 1)$ symmetric matrices over F_q . Suppose TurboBlom uses random matrices G and D , uniformly distributed over $\mathcal{G}$ and $\mathcal{D}$, respectively.

Then, for every integer $1 \leq l \leq \lambda$, we have

$$\text{fail}(l) \leq 1 - \left(1 - \frac{1}{|\mathcal{S}|^{\lambda+1-l}}\right)^2,$$

where $|\mathcal{S}|$ denotes the cardinality of the set $\mathcal{S}$.

Proof. Suppose that the adversary has captured l random nodes and extracted their secret keys. Since G is uniformly distributed over $\mathcal{G}$, we can w.l.g. assume that the nodes captured are $u_1, u_2, \dots, u_l$. Let u_s and u_d , $s, d > l$, be two random uncaptured nodes. Let $\mathbf{g}_i$ be the i th column vector of G , which corresponds to node u_i . By Theorem 2, the adversary gains no information about the pairwise key between u_s and u_d if and only if neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$.

Let $l', l' \leq l$, denote the maximum number of independent vectors among $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$. We can w.l.g. assume that vectors $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_{l'}$ are linearly independent. By performing elementary column operations, we can convert the $(\lambda + 1) \times l'$ matrix $[\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_{l'}]$ into a $(\lambda + 1) \times l'$ matrix that contains the identity matrix $I_{l'}$, that is $I_{l'}$ is a submatrix of H . Note that $\mathbf{g}_s$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_{l'}$ if and only if $\mathbf{g}_s$ is in the column vector space of matrix H , i.e.

$$\mathbf{g}_s = \sum_{i=1}^{l'} a_i \cdot \mathbf{h}_i, \quad (15)$$

where $\mathbf{h}_i$ denotes the i th column vector of H . We have $a_i \in \mathcal{S}$ in (15) because i) the elements of $\mathbf{g}_s$ are in $\mathcal{S}$, and ii) the identity matrix $I_{l'}$ is a submatrix of H .

Let

$$\mathcal{A} = \{\mathbf{g} | \mathbf{g} = \sum_{i=1}^{l'} a_i \cdot \mathbf{h}_i, \text{ for some } a_i \in \mathcal{S}\},$$

and

$$\mathcal{B} = \{\mathbf{g} | \mathbf{g} = [\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_{\lambda+1}]^T, \mathbf{g}_i \in \mathcal{S}\}.$$

The column vector $\mathbf{g}_s$ is uniformly distributed over $\mathcal{B}$. Moreover, Eq (15) holds if and only if $\mathbf{g}_s \in \mathcal{A}$. Therefore, the probability that (15) holds is equal to

$$\begin{aligned} \frac{|\mathcal{A} \cap \mathcal{B}|}{|\mathcal{B}|} &\leq \frac{|\mathcal{A}|}{|\mathcal{B}|} \\ &= \frac{|\mathcal{S}|^{l'}}{|\mathcal{S}|^{\lambda+1}} \\ &= \frac{1}{|\mathcal{S}|^{\lambda+1-l'}} \\ &\leq \frac{1}{|\mathcal{S}|^{\lambda+1-l}} \end{aligned}$$

Thus, the probability that $\mathbf{g}_s$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$ is at most $\frac{1}{|\mathcal{S}|^{\lambda+1-l}}$.

Similarly, we can show that probability that $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$ is at most $\frac{1}{|\mathcal{S}|^{\lambda+1-l}}$. Therefore, the probability that either $\mathbf{g}_s$ or $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$ is at most

$$1 - \left(1 - \frac{1}{|\mathcal{S}|^{\lambda+1-l}}\right)^2.$$

Example 4. Suppose TurboBlom uses a random generator matrix with elements from the set $\mathcal{S}$ defined in (5), over F_p , where p is the Mersenne prime $2^{127} - 1$. By Theorem 3, we have

$$\text{fail}(l) \leq 1 - \left(1 - \frac{1}{|\mathcal{S}|^{\lambda+1-l}}\right)^2.$$

By Remark (2), the set $\mathcal{S}$ has $2n + 1$ distinct elements, where $n = 127$. Therefore, $|\mathcal{S}| = 2n + 1 = 2 \cdot 127 + 1 = 255$. Thus, by Theorem 3, we get

$$\begin{aligned} \text{fail}(\lambda - 2) &\leq 1 - \left(1 - \frac{1}{|\mathcal{S}|^{\lambda+1-(\lambda-2)}}\right)^2 \\ &= 1 - \left(1 - \frac{1}{255^3}\right)^2 \\ &\approx 1.21 \cdot 10^{-7}. \end{aligned}$$

This shows that TurboBlom provides a high level of resilience even when $\lambda - 2$ nodes are captured. Recall that the Blom scheme is fully resilient up to λ node captures.

The case of zero-one matrices. By Theorem 3, TurboBlom can achieve a high resilient against node capture if $|\mathcal{S}|$ is large enough (e.g., $|\mathcal{S}| = 255$ as in Example 4). An interesting question is whether TurboBlom can still achieve a high level of resilience if it uses a random zero-one generator matrix (for which $|\mathcal{S}| = 2$). A positive answer means that we can further speed up TurboBlom. This is for two reasons. First, each zero in the generator matrix—which occurs with probability 50% in a random zero-one matrix with uniform distribution—eliminates not only one field multiplication but also one field addition (because adding a number to zero is trivial). Second, if we use a zero-one generator matrix, we do not even need shift operations as multiplication by 1 is trivial.

In the following, we show the positive news that zero-one matrices can result in a $\text{fail}(l)$ that is exponentially small in λ . Informally, TurboBlom equipped with a random zero-one generator matrix achieves a high resilience against node capture if λ is large enough (e.g., $\lambda \geq 30$).

Theorem 4. Let λ and n be positive integers, $\mathcal{G}$ be the set of all $(\lambda + 1) \times n$ zero-one matrices, and $\mathcal{D}$ be the set of all $(\lambda + 1) \times (\lambda + 1)$ symmetric matrices over F_p , where $p > (\lambda + 1)!$ is a prime number. Suppose TurboBlom uses random matrices G and D , uniformly distributed over $\mathcal{G}$ and $\mathcal{D}$, respectively.

Then, for every integer l , $1 \leq l \leq \min\{\lambda, n - 2\}$, we have

$$\text{fail}(l) \leq 2 \cdot \left(\frac{1}{2} + o(1)\right)^{\lambda+1} \quad (16)$$

Proof. Suppose the adversary has captured l , $l \leq \lambda$, random nodes and extracted their secret keys. Since G is uniformly distributed over $\mathcal{G}$, we can w.l.g. assume that the nodes captured are $u_1, u_2, \dots, u_l$. Let u_s and u_d be two random uncaptured nodes (i.e., $l < s, d \leq n$). Let $\mathbf{g}_i$ be the i th column vector of G (i.e., the column of G that corresponds to node u_i). By Theorem 2, the adversary gains no information about the pairwise key between u_s and u_d if and only if neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$.

Let $\mathcal{H}_c$, $1 \leq c \leq \lambda + 1$, denote the set of all $(\lambda + 1) \times c$ zero-one matrices. Let H be a random matrix uniformly distributed over $\mathcal{H}_{\lambda+1}$. Note that the matrix $G_s = [\mathbf{g}_s, \mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l]$ is a random

matrix uniformly distributed over H_{l+1} . We have

$$\begin{aligned} & \mathbb{P}(\mathbf{g}_s \text{ is a linear combination } \mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l) \\ & \leq \mathbb{P}(\text{Rank}(G_s) < l + 1) \\ & \leq \mathbb{P}(\text{Rank}(H) < \lambda + 1) \\ & = \mathbb{P}((\det H \bmod p) = 0) \end{aligned}$$

We have

$$\det H \leq (\lambda + 1)!$$

because H is a $(\lambda + 1) \times (\lambda + 1)$ zero-one matrix. Furthermore, by the theorem statement, we have $p > (\lambda + 1)!$. Therefore, we get

$$\det H < p,$$

thus

$$(\det H \bmod p) = 0 \Leftrightarrow \det H = 0.$$

Consequently

$$\begin{aligned} \mathbb{P}((\det H \bmod p) = 0) &= \mathbb{P}(\det H = 0) \\ &= \left(\frac{1}{2} + o(1)\right)^{\lambda+1}, \end{aligned}$$

where the second equation is by Theorem A of [32]. Therefore, the probability that $\mathbf{g}_s$ (similarly $\mathbf{g}_d$) is a linear combination of vectors $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$ is at most $\left(\frac{1}{2} + o(1)\right)^{\lambda+1}$. Thus, by a union bound, the probability that either $\mathbf{g}_s$ or $\mathbf{g}_d$ is a linear combination of vectors $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$ is at most $2 \cdot \left(\frac{1}{2} + o(1)\right)^{\lambda+1}$, hence

$$\text{fail}(l) \leq 2 \cdot \left(\frac{1}{2} + o(1)\right)^{\lambda+1}$$

Example 5. Let $\lambda + 1 = 30$, that is each node is preloaded with 30 secret keys. Let n be a positive integer, and $\mathcal{G}$ be the set of all $(\lambda + 1) \times n$ zero-one matrices over F_p , where p is a prime number. Let us set the security level to 160 bits (i.e., set the size of pairwise keys to 160 bits) by choosing p to a prime number of 160 bits, i.e. $2^{159} < p < 2^{160}$. Suppose TurboBlom uses a random zero-one generator matrix uniformly distributed over $\mathcal{G}$.

We have $p > (\lambda + 1)!$ in this example. Thus, by Theorem 4, we get that $\text{fail}(\lambda)$ is of order $\frac{1}{2^{30}}$. This implies that there is only a small chance for the adversary to compromise the communication between two nodes u_s and u_d by capturing λ other nodes in the network. Note that $\text{fail}(\lambda)$ is remarkably small in TurboBlom despite the use of a random zero-one matrix as the generator matrix.

In Theorem 4, we assumed that $p > (\lambda + 1)!$. This does not hold in all settings. For instance, as in Example 5, we may set the security level to 160 bits (i.e. use a prime p of size 160 bits), but preload nodes with 50 secret keys (instead of 30 as in Example 5). In this case we have $p < (\lambda + 1)!$ because $p < 2^{160} < 50!$. The next theorem addresses this issue by relaxing the assumption $p > (\lambda + 1)!$.

Theorem 5. Let λ and n be positive integers, $\mathcal{G}$ be the set of all $(\lambda + 1) \times n$ zero-one matrices, and $\mathcal{D}$ be the set of all $(\lambda + 1) \times (\lambda + 1)$ over F_p , where p is a random prime number of size t bits, that is $2^{t-1} < p < 2^t$ for an integer t . Suppose $t \geq 16$ (in practice, the value of t is set much larger

than 16). Suppose TurboBlom uses random matrices G and D , uniformly distributed over $\mathcal{G}$ and $\mathcal{D}$, respectively.

Then, for every integer l , $1 \leq l \leq \min\{\lambda, n-2\}$, we have

$$\text{fail}(l) \leq 2 \cdot \left\{ \left(\frac{1}{2} + o(1) \right)^{\lambda+1} + \frac{\lambda \cdot \log_2(\lambda+1)}{2^{t-1}} \right\}$$

Proof. Suppose the adversary has captured l random nodes and extracted their secret keys. Since G is uniformly distributed over $\mathcal{G}$, we can w.l.g. assume that the nodes captured are $u_1, u_2, \dots, u_l$. Let u_s and u_d be two random uncaptured nodes (i.e., $1 < s, d \leq n$). Let $\mathbf{g}_i$ be the i th column vector of G (i.e., the column of G that corresponds to node u_i). By Theorem 2, the adversary gains no information about the pairwise key between u_s and u_d if neither $\mathbf{g}_s$ nor $\mathbf{g}_d$ is a linear combination of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$, that is both $\mathbf{g}_s$ and $\mathbf{g}_d$ are linearly independent of $\mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l$.

Let $\mathcal{H}_c$, $1 \leq c \leq \lambda+1$, denote the set of all $(\lambda+1) \times c$ zero-one matrices. Let H be a random matrix uniformly distributed over $\mathcal{H}_{\lambda+1}$. Note that the matrix $G_s = [\mathbf{g}_s, \mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l]$ is a random matrix uniformly distributed over $\mathcal{H}_{l+1}$. We have

$$\begin{aligned} & \mathbb{P}(\mathbf{g}_s \text{ is a linear combination of } \mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l) \\ & \leq \mathbb{P}(\text{Rank}(G_s) < l+1) \\ & \leq \mathbb{P}(\text{Rank}(H) < \lambda+1) \\ & = \mathbb{P}((\det H \bmod p) = 0) \end{aligned}$$

Thus, by a union bound, we get

$$\begin{aligned} \text{fail}(l) &= \mathbb{P}(\mathbf{g}_s \text{ or } \mathbf{g}_d \text{ is a linear combination of } \mathbf{g}_1, \mathbf{g}_2, \dots, \mathbf{g}_l) \\ &\leq 2 \cdot \mathbb{P}((\det H \bmod p) = 0). \end{aligned} \tag{17}$$

Let $\mathcal{E}_1, \mathcal{E}_2$ and $\mathcal{E}_3$ be the following events:

$$\begin{aligned} \mathcal{E}_1 : & \quad \det H \bmod p = 0 \\ \mathcal{E}_2 : & \quad \det H = 0 \\ \mathcal{E}_3 : & \quad (\det H \neq 0) \wedge (p \mid \det H). \end{aligned}$$

We have

$$\mathcal{E}_1 = \mathcal{E}_2 \vee \mathcal{E}_3,$$

Therefore, by (17), we get

$$\begin{aligned} \text{fail}(l) &\leq 2 \cdot \mathbb{P}(\mathcal{E}_1) \\ &= 2 \cdot \mathbb{P}(\mathcal{E}_2 \vee \mathcal{E}_3) \\ &\leq 2 \cdot \mathbb{P}(\mathcal{E}_2) + 2 \cdot \mathbb{P}(\mathcal{E}_3), \end{aligned}$$

where the last inequality is by the union bound. By Theorem A of [32], we have

$$\mathbb{P}(\mathcal{E}_2) = \left(\frac{1}{2} + o(1)\right)^{\lambda+1}$$

Next, to conclude the proof, we show that

$$\mathbb{P}(\mathcal{E}_3) \leq \frac{\lambda \cdot \log_2(\lambda + 1)}{2^{t-1}}.$$

Suppose that $\det H$ is non-zero and has $k \geq 0$ different prime factors of size t bits. The product of k numbers each of size t bits is at least $(2^{t-1})^k = 2^{(t-1)k}$. Therefore, we must have

$$\det H \geq 2^{(t-1)k},$$

thus, by applying the log function on both sides of the above inequality, we get

$$k \leq \frac{\log_2(\det H)}{t-1}$$

Let ζ denote the number of prime numbers of size t bits. Since p is a random prime of size t bits, and $\det H$ has k different prime factors of size t bits, the probability that p is a factor of $\det H$ is

$$\begin{aligned} \mathbb{P}(p \mid \det H) &= \frac{k}{\zeta} \\ &\leq \frac{\log_2(\det H)}{\zeta(t-1)} \\ &\leq \frac{\log_2((\lambda+1)!)}{\zeta(t-1)} \\ &\leq \frac{\log_2((\lambda+1)^\lambda)}{\zeta(t-1)} \\ &\leq \frac{\lambda \log_2(\lambda+1)}{\zeta(t-1)} \end{aligned} \tag{18}$$

Due to Dusart [33], for any integer $x \geq 60184$, we have

$$\frac{x}{\ln x - 1} \leq \pi(x) \leq \frac{x}{\ln x - 1.1},$$

where $\pi(x)$ denotes the number of primes not greater than x . We have $2^{16} > 60184$. Therefore,

the number of primes of size t bits, $t \geq 16$, is

$$\begin{aligned}\zeta &= \pi(2^t) - \pi(2^{t-1}) \\ &\geq \frac{2^t}{\ln(2^t) - 1} - \frac{2^{t-1}}{\ln(2^{t-1}) - 1.1} \\ &= \frac{2^t}{t \cdot \ln 2 - 1} - \frac{2^{t-1}}{(t-1) \cdot \ln 2 - 1.1} \\ &\geq \frac{2^{t-1}}{t-1}.\end{aligned}$$

Therefore, by (18) we get

$$\frac{k}{\zeta} \leq \frac{\lambda \log_2(\lambda + 1)}{2^{t-1}},$$

hence

$$\mathbb{P}(\mathcal{E}_3) = \frac{k}{\zeta} \leq \frac{\lambda \cdot \log_2(\lambda + 1)}{2^{t-1}},$$

as needed.

5 A sample application of TurboBlom

To showcase an application of TurboBlom, we use the scheme in this section to present an efficient way for end-to-end sender authentication in RPL. We start with covering the basics of the RPL standard, and explaining our system model.

Basics of RPL

RPL is a standard routing protocol for LLNs. RPL supports three types of traffic patterns; Point-to-Multipoint (P2MP), Multipoint-to-point (MP2P), and Point-to-Point (P2P). To support these traffic patterns, RPL builds one or more Destination Oriented Directed Acyclic Graph (DODAG) each of which is uniquely identified by a set of identifier consisting DODAG ID, instance ID, and version number (Fig 6). Each DODAG has a specific node, called root, that acts as a sink for other nodes in DODAG and connect the DODAG to other network. To create and maintain the DODAG RPL uses three different control messages; DODAG Information Object (DIO), DODAG Information Solicitation (DIS), and Destination Advertisement Object (DAO). RPL control messages are ICMPv6 message with type equal to 155. The root initiates and manages the DODAG by broadcasting DIO messages over the network.

Prior to joining the network, each node listens for DIO messages. When a node receives a DIO, it can join the network. After joining the network, a node continuously advertises DIO to indicate its presence. A node can send DIS to accelerate joining the network or to recover from topology errors. RPL uses DAO to propagate nodes' information toward the root and fill nodes' routing tables in support of P2MP and P2P traffic.

RPL may operate in one of the following security modes:

- *unsecured mode*: In this mode, RPL does not provide a security measure. It may, however, utilize the link-layer security to protect its messages.
- *pre-installed mode*: In this mode, RPL messages are protected using pre-defined security mechanisms. secret keys are installed on nodes before they are deployed in the network. Nodes will use these preloaded keys to secure RPL messages.

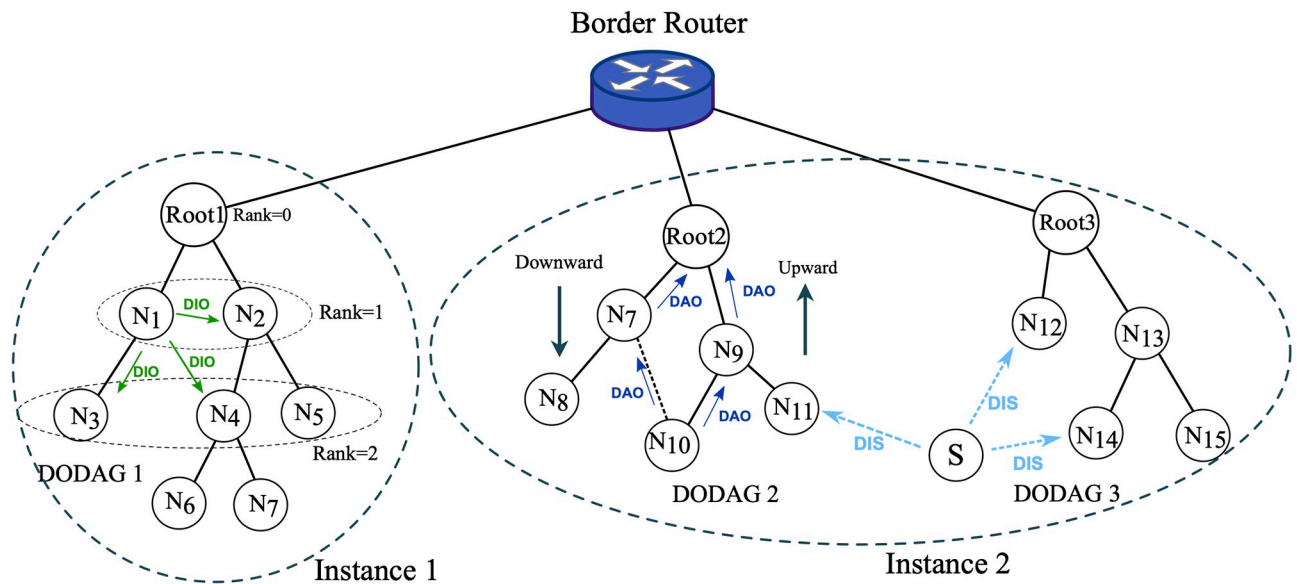


Fig 6. An example of RPL network with two instances and three DODAGs.

<https://doi.org/10.1371/journal.pone.0295190.g006>

- *authenticated mode*: similar to the previous mode, nodes have pre-installed keys, but these keys are used only for authenticating the nodes who want to join the network as a leaf. All nodes except the leaf nodes must obtain a second key from an authentication authority after joining the network.

The general format of an RPL control message is shown in Fig 7. The ICMPv6 code field dictates the type of RPL control message: DIO, DIS, or DAO. The Algorithm field determines the algorithm used for authentication/encryption, and the LVL bits specify whether the message is only authenticated, encrypted, or both.

Sender authentication problem in RPL

Sender authentication may be provided using symmetric-key or public-key cryptography. With public-key cryptography (via signatures), a sender can be uniquely identified, whereas with symmetric-key-based techniques, sender authenticity is only provided relative to devices in a key-sharing group. Therefore, in the absence of computationally-heavy public-key-based techniques, RPL is unable to provide fine-grained authentication, which is needed in scenarios that require group communications or in scenarios that require non-repudiation.

Many recent works have attempted to mitigate the sender's authentication problem (or closely related problems such as the Sybil attack) in RPL networks. In a recent work, for instance, Raoof *et al.* [24] proposed a new secure mode, Chained Secure Mode (CSM), for the RPL protocol. In CSM, each node encodes its outgoing messages using a randomly generated code, and updates its neighbors with the next code. CSM only provides sender authentication between neighbors, requires a recovery mechanism, assumes that every first message truly comes from the original sender, and uses a default value of zero as the secret code for the first message from any sender. With regards to Sybil attack, current solutions (e.g., [25–29]) offer limited mitigation, can only detect a special kind of the attack, or solely detect the existence of Sybil attack in the network.

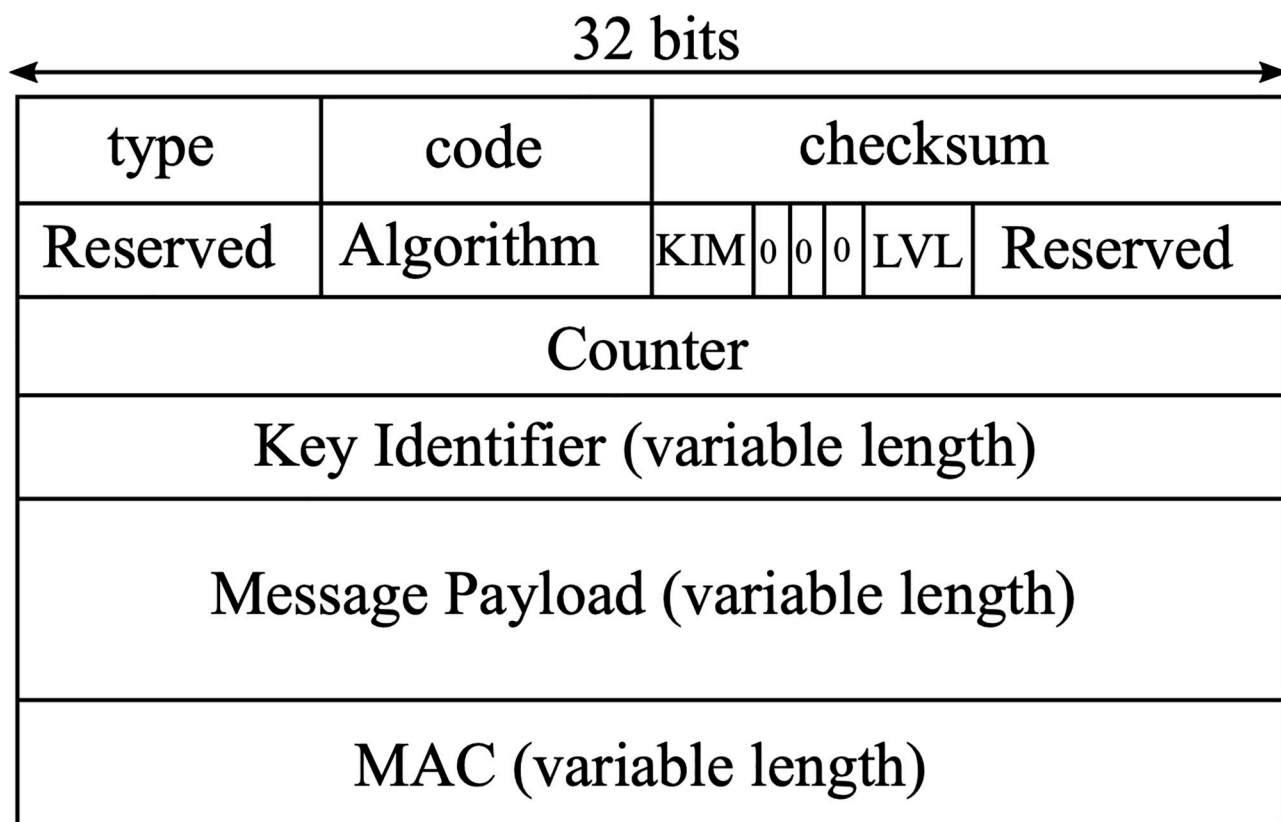


Fig 7. Secured ICMPv6 message format.

<https://doi.org/10.1371/journal.pone.0295190.g007>

To provide sender authentication in RPL, and prevent Sybil attacks (even in the presence of small-scale capture attacks), we propose a new lightweight TurboBlom-based solution in this section. A core feature of our solution is that it prevents an adversary, with high probability, to claim the identity of an uncaptured node even when the adversary has captured and extracted the secret keys of up to λ nodes, where λ is a security parameter. We formally state this feature in Proposition 6.

In contrast to the secure CSM mode [24], our solution

- provides end-to-end sender authentication (i.e., communicating nodes do not have to be neighbors);
- does not assume that the first message is truly from the original sender;
- does not require a recovery mechanism when a message is lost (message loss can happen frequently in lossy networks such as LLNs);
- is non-interactive: a sender does not require prior interactions with a receiver in order to send an authenticated message to the receiver.

System model

We consider an RPL network, which can support up to n nodes. We assume that there is a trusted entity that, prior to deployment, preloads each node with a set of $\lambda + 1$ secret keys. The

trusted entity also preloads every node with public information including an IP address from a block of IP addresses $[IP_{min}, IP_{max}]$ with n IP addresses (i.e. $IP_{max} - IP_{min} + 1 = n$), as well as the range of the block (i.e., IP_{min} and IP_{max}).

We consider an adversary who has captured and extracted the secret keys of l , $l \leq \lambda$, random nodes. The adversary can also be viewed as a coalition of l random malicious nodes. Our objective is to design a sender-authentication method with high resilience to the above adversary as defined below.

Definition 3 (A Resilient Sender-Authentication Method). Let $0 \leq l \leq \lambda$ be an integer, $\mathcal{L} \subset [IP_{min}, IP_{max}]$ be a set of $l = |\mathcal{L}|$ IP addresses selected uniformly at random, and IP_c be an IP address selected uniformly at random from $[IP_{min}, IP_{max}] \setminus \mathcal{L}$. Consider an adversary who has captured and extracted the secret keys of all the nodes with IP addresses in $\mathcal{L}$. Given a sender-authentication method, let $\text{masq}(l)$ denote the probability that the adversary is able to masquerade as IP_c . Then, we call the method a resilient sender-authentication method if $\text{masq}(l)$ is negligible, more specifically if $\text{masq}(l) \leq c^\lambda$, for a fixed constant $0 < c < 1$.

TAM: A TurboBlom-based authentication mode

In this section, we introduce TAM, a lightweight TurboBlom-based pre-installed mode that provides sender authentication in RPL and is resilient to capture attacks. First, we explain how it works before nodes are deployed in the network.

Prior to deployment. Before nodes are deployed in the network, the trusted entity generates a random public generator matrix G and a random symmetric private matrix D over a finite field F_q of large order. To generate the random matrix G , the trusted entity uses a publicly-known random number generator (e.g., a cryptographic hash function). Since the used random number generator is publicly known, anyone can derive the column of G assigned to an IP address by applying the random number generator function to the IP address. This brings two advantages. First, it binds a column of G to an IP address. Second, nodes do not require to store any column of G (hence can save storage) as each column of G can be simply derived using the random number generator.

After deployment. Suppose that node u_i with IP address IP_i wants to send an authenticated message to another node u_j , with IP address IP_j . At the high level, u_i first computes $G_{:,j}$ (the public information of u_j) from IP_j by applying the random number generator to IP_j , that is

$$G_{:,j} = H(IP_j), \quad (19)$$

where the function $H(\cdot)$ represents the random number generator. It then uses the inner product (3) to compute the pairwise key $K_{i,j}$. After computing $K_{i,j}$, u_i generates a HMAC of its message using the key $K_{i,j}$ and concatenate the MAC to the message. Note the the sender does not require to interact with the receiver to obtain $G_{:,j}$ because the receiver can simply calculate $G_{:,j}$ from the IP address of the sender (i.e. from IP_j) using (19).

The following steps explain the above procedure in more details (see also Fig 8).

1. Prepare the message as per RPL's pre-installed mode;
2. Generate $G_{:,j}$ using the IP address of u_j ;
3. Compute the pairwise key $K_{i,j}$ using $G_{:,j}$ and $A_{i,:}$, where $A_{i,:}$ is the vector of u_i 's secret keys. At this step, u_i can optionally cache $K_{i,j}$ for later communications with u_j ;
4. Set the LVL and Algorithm fields in the header of ICMPv6 message (Fig 7) to reflect that TAM authentication is enabled;

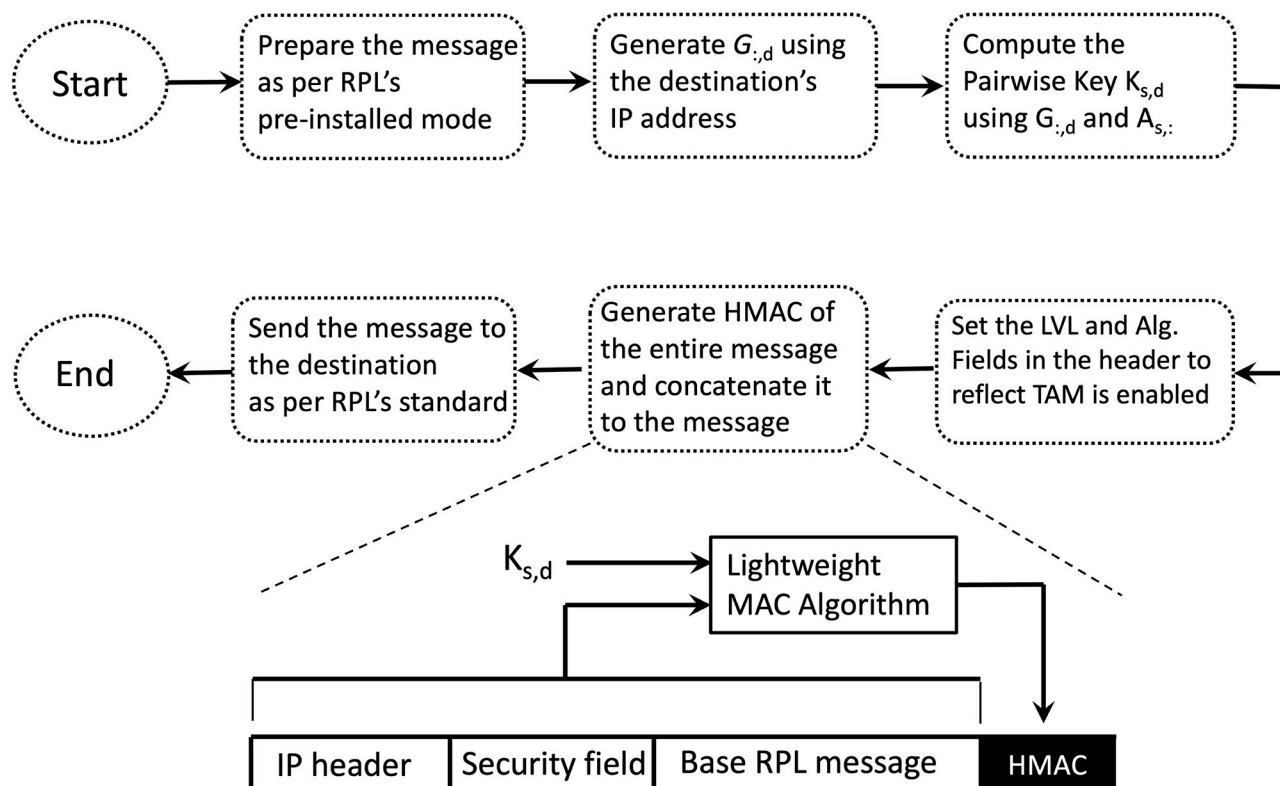


Fig 8. Sending a TAM authenticated packet from a source node (s) to a destination (d).

<https://doi.org/10.1371/journal.pone.0295190.g008>

5. Generate the HMAC of the entire message

$$\text{HMAC} = \text{Hash}(K_{i,j}, \text{message}),$$

using a proper lightweight MAC algorithm such as light-MAC [34], and concatenate the MAC to the message;

6. Send the packet to the destination u_j as per the RPL standard.

At the receiver side, node u_j first checks the header of the received message. If the TAM mode is enabled, u_j checks if the sender's IP address (which exists in the header of the received packet) is within the range $[IP_{min}, IP_{max}]$. If so, u_j generates $G_{s,i}$ using the IP address of the sender. It then computes the pairwise key, $K_{j,i} = K_{i,j}$, generates the HMAC of the received message, and compares it to the HMAC embedded in the message. If the two HMACs are equal, u_j accepts the message as authenticated; otherwise, u_j drops the message. The above procedure is illustrated in Fig 9. Note that the procedure is non-interactive, does not assume that the first message truly comes from the sender, and does not assume that the sender and receiver are neighbors.

The following proposition shows that TAM is a resilient sender-authentication method as defined in Definition 3.

Proposition 6. TAM is a resilient sender-authentication method.

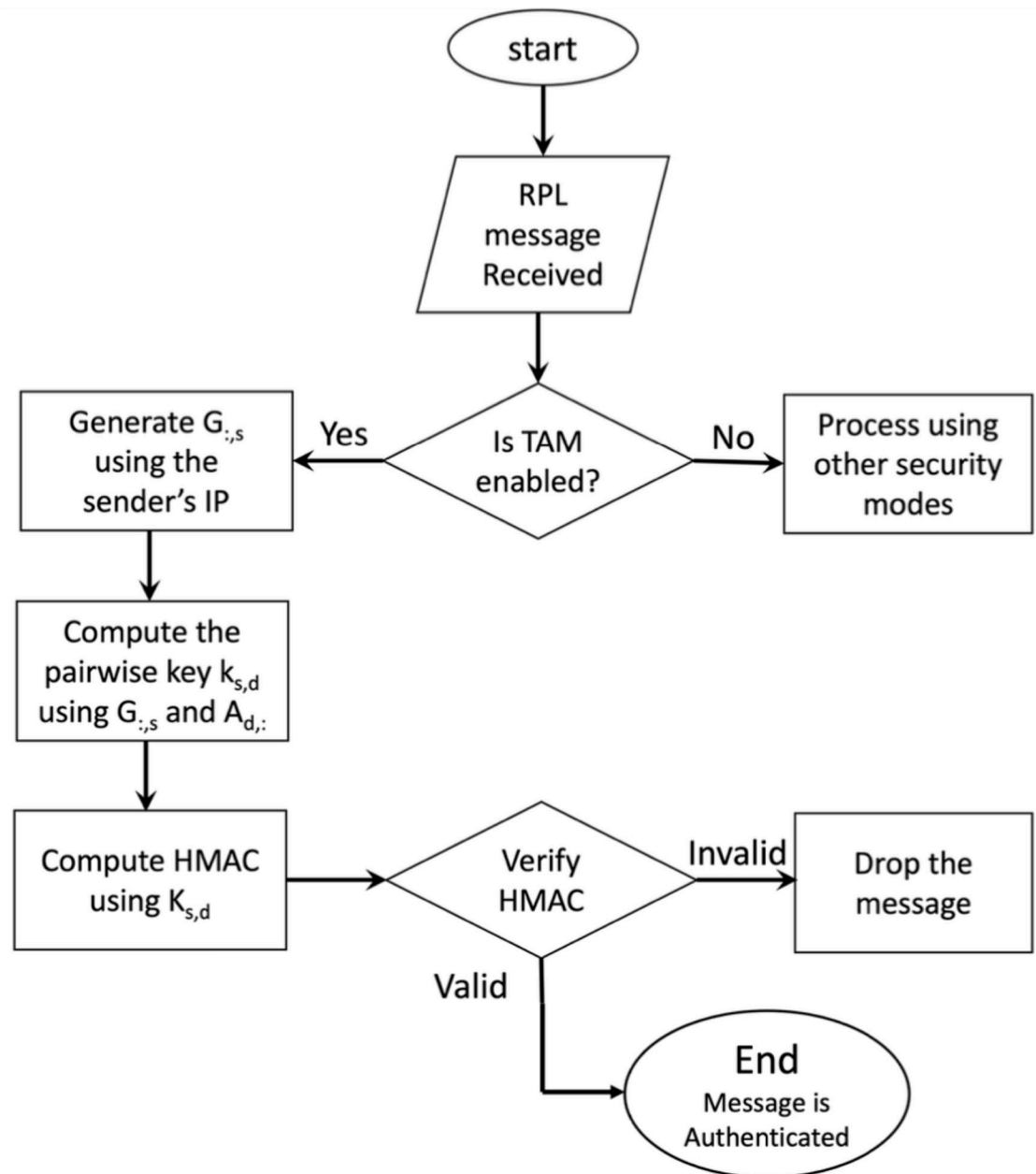


Fig 9. The procedure of receiving an authenticated packet in a destination node (d) from a source node (s).

<https://doi.org/10.1371/journal.pone.0295190.g009>

Proof. Let G denote the generator matrix of the TurboBlom used in TAM. We have

$$\begin{aligned}
 \text{fail}(I) &= 1 - (1 - \epsilon)^2 \\
 &= 2\epsilon - \epsilon^2 \\
 &\geq \epsilon,
 \end{aligned} \tag{20}$$

where ϵ is the probability that the column of G corresponding to IP_c is a linear combination of the columns of G that correspond to the IP addresses in $\mathcal{L}$ (IP_c and $\mathcal{L}$ are defined in Definition

3). By definition, we have $\text{masq}(l) = \epsilon$, thus substituting ϵ with $\text{masq}(l)$ in (20) we get

$$\text{masq}(l) \leq \text{fail}(l). \quad (21)$$

Therefore, if we use a random zero-one matrix G , by Theorem 4 we get $\text{fail}(l) \leq c^\lambda$ for a constant $0 < c < 1$, which by (21) implies $\text{masq}(l) \leq c^\lambda$ for a constant $0 < c < 1$.

Example 6. Consider a network which can support up to 1024 nodes (i.e., the network has been allocated a block with 1024 IP addresses). Suppose we use TurboBlom equipped with a random zero-one generator matrix. Let $\lambda + 1 = 50$, that is each node is pre-loaded with 50 secret keys. By Theorems 4 and 5, $\text{fail}(l) \approx \frac{1}{2^{50}}$ for any $l \leq \lambda$. Therefore, by the union bound, the probability that an adversary who has captured $l < 50$ random nodes is able to masquerade as any valid IP in the block of 1024 IPs (other than those 50 IP addresses that belong to the captured node) is at most

$$\begin{aligned} (1024 - 50) \cdot \text{masq}(l) &\leq 1024 - 50 \cdot \text{fail}(l) \\ &\approx 10^{-13} \end{aligned}$$

for any $l \leq \lambda$. Therefore, an adversary who has captured less than 50 random nodes has an extremely low chance ($\approx 10^{-13}$) of being able to masquerade as any valid IP address that does not belong to the captured nodes.

6 A numerical case study

To confirm our analytical results, we evaluated the resilience of TurboBlom in a practical setting, where we assume a network with up to $n = 1000$ devices, each capable of storing $\lambda + 1 = 30$ secret keys. We set the order of the finite field to 160 bits, and constructed the generator matrix G randomly by choosing its elements from $\mathcal{S} = \{-1, 0, 1\}$ uniformly at random. The choice of $\mathcal{S}$ is because it reduces all operations in the key generation into simple additions and subtractions, yet it provides higher resilience than the case $\mathcal{S} = \{0, 1\}$. In simple words, the use of $\mathcal{S} = \{-1, 0, 1\}$ in practice is more justified than the use of $\mathcal{S} = \{0, 1\}$, as the former provides higher resilience at virtually the same computational cost as the latter. The simulation setting parameters are summarized in Table 2.

We simulated the capturing attack by selecting $\lambda = 29$ columns of G uniformly at random and checking if the capture of these 29 nodes (corresponding to the selected columns) would compromise the key of any of the uncaptured nodes. We repeated the above process 100,000 times. The above numerical analysis took more than 240 hours on a Microsoft Azure server with 64 virtual CPUs and 512 GB RAM, yet it was unable to find any set of 29 nodes whose capture results in the compromise of a pairwise key between uncaptured nodes. This is in line with our analytical results which showed that the chance of an adversary to compromise the pairwise key between two uncaptured nodes is extremely low when the adversary captures up to $\lambda = 29$ random nodes.

Table 2. Simulation setting parameters.

Parameter	Value	Rationale
No. of devices	$n \leq 1000$	Covers most applications
No. of stored keys	$\lambda + 1 = 30$	A small value of λ to showcase the high resilience of TurboBlom.
Order of finite field	160 bits	To set the security bits to 160 bits
The set $\mathcal{S}$	$\mathcal{S} = \{-1, 0, 1\}$	Provides higher resilience than $\mathcal{S} = \{0, 1\}$ at virtually the same computational cost

<https://doi.org/10.1371/journal.pone.0295190.t002>

7 Conclusion and future work

In this paper, we introduced TurboBlom, a variant of the well-known Blom key predistribution scheme. TurboBlom is significantly lighter than the Blom scheme as it does not require any field multiplications. Yet, similar to the Blom scheme, TurboBlom 1) guarantees that any two nodes can generate a pairwise key, and 2) is highly resilient against node capture attack. This is notable as the Blom scheme provides the highest level of resilience against small-scale node capture attacks. The above features of TurboBlom make it suitable for computationally-constrained devices which exist in abundance in IoT, for instance as part of RPL networks. Finally, to showcase TurboBlom, we proposed TAM, a lightweight and TurboBlom-based secure mode that enables sender authentication in RPL networks. The proposed authentication method is highly resilient to node capture/collusion thanks to the proven resilience of TurboBlom.

One interesting future research direction is to improve the result of Theorem 3. Theorem 3 already establishes a lower bound on the resilience of the TurboBlom. By Theorems 4 and 5, we believe that there is substantial room for enhancing this lower bound. Another interesting direction is to analyze node capture attacks in a setting where the attacker can selectively, rather than randomly (as typically done in the literature), capture nodes.

Author Contributions

Data curation: Majid Khabbazzian.

Formal analysis: Majid Khabbazzian.

Funding acquisition: Majid Khabbazzian.

Investigation: Ahmad Shabani-Baghani.

Methodology: Majid Khabbazzian, Reihaneh Safavi-Naini.

Project administration: Majid Khabbazzian.

Supervision: Majid Khabbazzian, Reihaneh Safavi-Naini.

Validation: Ahmad Shabani-Baghani.

Writing – original draft: Majid Khabbazzian.

Writing – review & editing: Majid Khabbazzian.

References

1. Zhou W, Jia Y, Yao Y, Zhu L, Guan L, Mao Y, et al. Discovering and Understanding the Security Hazards in the Interactions between IoT Devices, Mobile Apps, and Clouds on Smart Home Platforms. In: Heninger N, Traynor P, editors. 28th USENIX Security Symposium; 2019. p. 1133–1150.
2. Farrukh H, Ozmen M, Ors F, Celik Z. One Key to Rule Them All: Secure Group Pairing for Heterogeneous IoT Devices. In: IEEE Symposium on Security and Privacy (S&P); 2023. p. 3026–3042.
3. Bormann C, Ersue M, Keränen A. Terminology for Constrained-Node Networks. RFC 7228. 2014; p. 1–17.
4. Shelby Z, Hartke K, Bormann C. The Constrained Application Protocol (CoAP). RFC 7252. 2014; p. 1–112.
5. Sun F, He S, Zhang X, Zhang J, Li Q, He Y. A Fully Authenticated Diffie-Hellman Protocol and Its Application in WSNs. IEEE Trans Inf Forensics Secur. 2022; 17:1986–1999. <https://doi.org/10.1109/TIFS.2022.3173536>
6. Singh KR, Das P, Banerjee S. Nested block design as key pre-distribution in wireless sensor networks. Discret Math Algorithms Appl. 2022; 14(1):1–12. <https://doi.org/10.1142/S1793830921501007>

7. Zhou B, Li S, Wang J, Cheng Y, Wu J. A secure model against mobile sink replication attacks in unattended sensor networks. *Computer Networks*. 2023; 221:109529. <https://doi.org/10.1016/j.comnet.2022.109529>
8. Eschenauer L, Gligor VD. A key-management scheme for distributed sensor networks. In: *Proceedings of the 9th ACM Conference on Computer and Communications Security*; 2002. p. 41–47.
9. Chan H, Perrig A, Song D. Random key predistribution schemes for sensor networks. In: *2003 Symposium on Security and Privacy*, 2003. IEEE; 2003. p. 197–213.
10. Blom R. An optimal class of symmetric key generation systems. In: *Workshop on the Theory and Application of Cryptographic Techniques*. Springer; 1984. p. 335–338.
11. Alexander R, Brandt A, Vasseur J, Hui J, Pister K, Thubert P, et al. RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks; 2012. RFC 6550.
12. Mayzaud A, Badonnel R, Chrisment I. A Taxonomy of Attacks in RPL-based Internet of Things. *International Journal of Network Security*. 2016; 18(3):459–473.
13. Carman DW, Kruus PS, Matt BJ. Constraints and approaches for distributed sensor network security. NAI Labs Technical Report 00-010, <http://downloadnaicom>, Technical report. 2000; p. 1–139.
14. Blundo C, Santis AD, Herzberg A, Kutten S, Vaccaro U, Yung M. Perfectly-Secure Key Distribution for Dynamic Conferences. In: *Advances in Cryptology—CRYPTO*, 12th. vol. 740; 1992. p. 471–486.
15. Lee J, Stinson DR. Deterministic Key Predistribution Schemes for Distributed Sensor Networks. In: *Selected Areas in Cryptography*. vol. 3357 of *Lecture Notes in Computer Science*. Springer; 2004. p. 294–307.
16. Çamtepe SA, Yener B. Combinatorial design of key distribution mechanisms for wireless sensor networks. *IEEE/ACM Trans Netw*. 2007; 15(2):346–358. <https://doi.org/10.1109/TNET.2007.892879>
17. Paterson MB, Stinson DR. A unified approach to combinatorial key predistribution schemes for sensor networks. *Des Codes Cryptogr*. 2014; 71(3):433–457. <https://doi.org/10.1007/s10623-012-9749-4>
18. Esfehiani NS, Javadi HHS. A High Resilience and High Scalable Key Pre-distribution Scheme Based on Combinatorial Designs for Resource-Constrained Devices in IoT Networks. *Wirel Pers Commun*. 2022; 127(4):2905–2934. <https://doi.org/10.1007/s11277-022-09903-8>
19. Kittur LJ, Pais AR. Combinatorial Design Based Key Pre-distribution Scheme with High Scalability and Minimal Storage for Wireless Sensor Networks. *Wirel Pers Commun*. 2023; 128(2):855–873. <https://doi.org/10.1007/s11277-022-09979-2>
20. Antony SNFMA, Bahari MFA. Implementation of Elliptic Curves in the Polynomial Blom Key Pre-Distribution Scheme for Wireless Sensor Networks and Distributed Ledger Technology. *J Sens Actuator Netw*. 2023; 12(1):15. <https://doi.org/10.3390/jsan12010015>
21. Liu D, Ning P. Establishing pairwise keys in distributed sensor networks. In: *ACM Conference on Computer and Communications Security, CCS*. ACM; 2003. p. 52–61.
22. Du W, Deng J, Han YS, Varshney PK, Katz J, Khalili A. A pairwise key predistribution scheme for wireless sensor networks. *ACM Transactions on Information and System Security (TISSEC)*. 2005; 8(2):228–258. <https://doi.org/10.1145/1065545.1065548>
23. Lee J, Stinson DR. On the Construction of Practical Key Predistribution Schemes for Distributed Sensor Networks Using Combinatorial Designs. *ACM Trans Inf Syst Secur*. 2008; 11(2):1–135.
24. Raouf A, Matrawy A, Lung CH. Securing RPL using Network Coding: The Chained Secure Mode (CSM). *IEEE Internet of Things Journal*. 2021;.
25. Airehrour D, Gutiérrez JA, Ray SK. *SecTrust-RPL: A secure trust-aware RPL routing protocol for Internet of Things*. *Future Gener Comput Syst*. 2019; 93:860–876. <https://doi.org/10.1016/j.future.2018.03.021>
26. Medjek F, Tandjaoui D, Abdmeziem MR, Djedjig N. Analytical evaluation of the impacts of Sybil attacks against RPL under mobility. In: *2015 12th International Symposium on Programming and Systems (ISPS)*. IEEE; 2015. p. 1–9.
27. Murali S, Jamalipour A. A Lightweight Intrusion Detection for Sybil Attack Under Mobile RPL in the Internet of Things. *IEEE Internet of Things Journal*. 2020; 7(1):379–388. <https://doi.org/10.1109/JIOT.2019.2948149>
28. Pu C. Sybil Attack in RPL-Based Internet of Things: Analysis and Defenses. *IEEE Internet of Things Journal*. 2020; 7(6):4937–4949. <https://doi.org/10.1109/JIOT.2020.2971463>
29. Kaliyar P, Jaballah WB, Conti M, Lal C. LiDL: Localization with early detection of sybil and wormhole attacks in IoT Networks. *Comput Secur*. 2020; 94:101849. <https://doi.org/10.1016/j.cose.2020.101849>
30. Gouvêa CP, Oliveira LB, López J. Efficient software implementation of public-key cryptography on sensor networks using the MSP430X microcontroller. *Journal of Cryptographic Engineering*. 2012; 2(1):19–29. <https://doi.org/10.1007/s13389-012-0029-z>

31. de Clercq R, Uhsadel L, Van Herrewege A, Verbauwhede I. Ultra low-power implementation of ECC on the ARM Cortex-M0+. In: 2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC); 2014. p. 1–6.
32. Tikhomirov K. Singularity of random Bernoulli matrices. *Annals of Mathematics*. 2020; 191:593–634. <https://doi.org/10.4007/annals.2020.191.2.6>
33. Dusart P. Estimates of some functions over primes without R. H. preprint arxiv:10020442. 2010;.
34. Luykx A, Preneel B, Tischhauser E, Yasuda K. A MAC mode for lightweight block ciphers. In: International Conference on Fast Software Encryption. Springer; 2016. p. 43–59.